

40 PAGES
COMPLETE
HANDBOOK

VERIQTA
ENGINEERING EXCELLENCE

✓ DIAGNOSE
✓ ANALYZE
✓ RESOLVE
✓ RECOVER
✓ PREVENT

AWS



TROUBLESHOOTING HANDBOOK

FROM FUNDAMENTALS TO ENTERPRISE
PRODUCTION INCIDENT RESPONSE



TROUBLESHOOT
Identify issues
faster



ANALYZE
Understand root
causes



RESOLVE
Fix problems
effectively

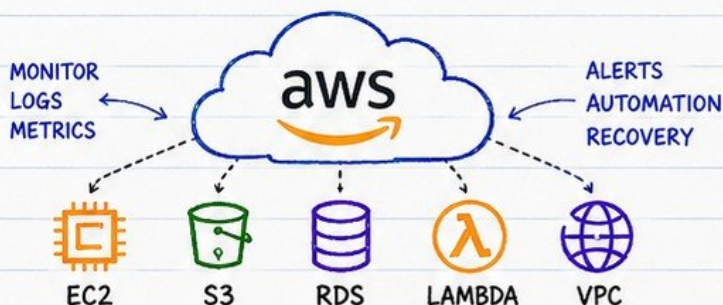


RECOVER
Restore services
reliably



PREVENT
Build resilient
systems

- ★ 40 Practical, Hands-on Topics
- ★ Real Production Scenarios
- ★ Step-by-Step Troubleshooting
- ★ Labs and Playbooks
- ★ Tools, Techniques & Checklists
- ★ Enterprise Best Practices
- ★ Senior Engineer Insights



SOLVE PROBLEMS. REDUCE DOWNTIME. IMPROVE RELIABILITY.
THAT'S HOW WE SHIP RELIABILITY.

VERIQTA SOCIAL



YouTube
@veriqta



GitHub
@veriqta



LinkedIn
@veriqta



X
@veriqta



Instagram
@veriqta



Telegram
@veriqta

1. INTRODUCTION TO AWS TROUBLESHOOTING

WHY TROUBLESHOOTING MATTERS

- ✓ Minimize downtime and service disruption
- ✓ Protect user experience and business revenue
- ✓ Maintain security, compliance, and reliability
- ✓ Reduce MTTR (Mean Time To Resolution)
- ✓ Improve system stability and performance
- ✓ Build trust with customers and stakeholders



PRODUCTION INCIDENT LIFECYCLE



AWS SHARED RESPONSIBILITY MODEL

aws AWS (RESPONSIBLE FOR)	CUSTOMER (RESPONSIBLE FOR)
- Global Infrastructure - Physical Security - Hardware, Software, Networking - Managed Services (e.g., RDS, S3) - Availability, Durability, Scalability	- Data and Content - Identity and Access Management - Operating Systems - Applications and Code - Network Configuration - Monitoring and Logging - Backups and Disaster Recovery



Security **IN** the Cloud is your responsibility.
Security **OF** the Cloud is AWS's responsibility.



TROUBLESHOOTING METHODOLOGY

- 1 Understand the Problem**
Collect information and reproduce the issue.
- 2 Gather Data**
Check metrics, logs, events, and configurations.
- 3 Analyze and Isolate**
Identify the component and root cause.
- 4 Implement Fix**
Apply the correct fix or workaround.
- 5 Validate and Monitor**
Confirm resolution and watch for recurrence.
- 6 Document and Improve**
Record findings and improve systems.



ROOT CAUSE ANALYSIS (RCA)

- ✓ Look beyond the symptoms
- ✓ Ask "Why?" repeatedly (5 Whys)
- ✓ Identify the underlying cause
- ✓ Determine contributing factors
- ✓ Implement preventive actions
- ✓ Track lessons learned



INCIDENT SEVERITY LEVELS

SEV 1	Critical	Major outage, no workaround, business impact is severe.
SEV 2	High	Partial outage or major degradation, urgent attention.
SEV 3	Medium	Minor impact, workaround available.
SEV 4	Low	Minimal impact, low urgency.



HANDBOOK OBJECTIVES

- ✓ Build a strong troubleshooting mindset
- ✓ Learn AWS service troubleshooting from fundamentals to advanced
- ✓ Master real-world production debugging
- ✓ Reduce MTTR and prevent repeat issues
- ✓ Prepare for enterprise production incident response
- ✓ Become a confident, senior-level AWS engineer



Great troubleshooting is a skill.
Practice, observe, analyze, document, and improve every day.

VERIQTA SOCIAL



YouTube
@veriqta



GitHub
@veriqta



LinkedIn
@veriqta



X
@veriqta



Instagram
@veriqta



Telegram
@veriqta

2. AWS MONITORING FUNDAMENTALS



Amazon CloudWatch is AWS's native monitoring and observability service. It collects, monitors, and analyzes metrics, logs, events, and alarms in real time.

AMAZON CLOUDWATCH CORE COMPONENTS

1



METRICS

Numeric data points collected over time. Monitor CPU, memory, requests, errors, latency, and more.

2



LOGS

Collect, store, and search log data from AWS services, applications, and systems.

3



DASHBOARDS

Visualize metrics and logs in one or more custom dashboards for real-time insights.

4



ALARMS

Set thresholds on metrics and get notified or trigger actions when conditions are breached.

5



EVENTS

Stream real-time AWS service events and take action using rules and targets.

6



INSIGHTS

CloudWatch Insights helps you analyze metrics and logs to detect anomalies and troubleshoot.

HOW CLOUDWATCH WORKS



AWS RESOURCES
EC2, RDS, S3, Lambda, VPC, ECS, EKS, etc.



CLOUDWATCH COLLECTS
Metrics, logs, events, and performance data



STORE & ANALYZE
Data is stored and analyzed in CloudWatch



ALERT & ACT
Alarms trigger notifications or automated actions

OBSERVE → ANALYZE → ALERT → RESOLVE → IMPROVE

COMMON USE CASES

- ✓ Monitor application and infrastructure health
- ✓ Detect and alert on performance issues
- ✓ Troubleshoot production incidents
- ✓ Track capacity and resource utilization
- ✓ Analyze logs for errors and anomalies
- ✓ Audit and compliance monitoring
- ✓ Automate remediation with events and alarms

MONITORING BEST PRACTICES

- ✓ Monitor everything that matters.
- ✓ Use meaningful metric names and dimensions.
- ✓ Set appropriate alarm thresholds.
- ✓ Use composite alarms to reduce noise.
- ✓ Enable detailed monitoring for critical resources.
- ✓ Centralize logs and use retention policies.
- ✓ Create dashboards for quick visibility.
- ✓ Automate responses using Events and Alarms.
- ✓ Regularly review and optimize monitoring setup.
- ✓ Test alarms and notifications periodically.



TIP

Good monitoring reduces downtime, improves performance, and builds operational confidence.

VERIQTA SOCIAL



YouTube
@veriqta



GitHub
@veriqta



LinkedIn
@veriqta



X
@veriqta



Instagram
@veriqta



Telegram
@veriqta

3. CLOUDTRAIL AND AWS CONFIG



Amazon CloudTrail records API activity and user events.
 AWS Config records configuration changes and evaluates compliance.
 Together, they provide complete visibility, accountability, and governance.



A. CLOUDTRAIL OVERVIEW

- ✓ CloudTrail records all API calls and account activity.
- ✓ Captures management events, data events, and insights.
- ✓ Helps with security analysis, troubleshooting, and compliance.
- ✓ Logs are delivered to S3, CloudWatch Logs, or EventBridge.



EVENT HISTORY



- View recent events in the CloudTrail console.
- Filter by time, user, event name, resource, IP address, and more.
- Investigate who did what, when, and from where.
- Export logs for deeper analysis and audits.

EVENT TYPES

Type	Description
Management Events	Operations that write or read AWS resources (default).
Data Events	Detailed events for S3, Lambda, DynamoDB, and more.
Insights Events	Detects unusual API activity (anomalies).
Network Activity Events	Captures network activity for supported resources.

B. AWS CONFIG OVERVIEW

- ✓ AWS Config tracks configuration changes of AWS resources.
- ✓ Maintains a history of resource configurations over time.
- ✓ Evaluates configurations against desired settings.
- ✓ Provides visibility into resource relationships and dependencies.



CONFIGURATION HISTORY

- View configuration snapshots of resources.
- See when a change occurred and what changed.
- Compare configuration items over time.
- Helps in rollback, compliance, and auditing.



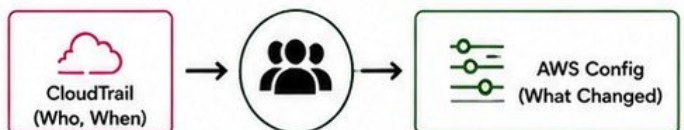
COMPLIANCE TRACKING

- ✓ Define AWS Config Rules to evaluate configurations.
- ✓ Rules are based on AWS best practices or custom logic.
- ✓ Resources are marked as COMPLIANT or NON_COMPLIANT.
- ✓ Supports remediation with AWS Systems Manager.



CHANGE DETECTION

- ✓ CloudTrail shows WHO made the change and WHEN.
- ✓ AWS Config shows WHAT changed and the before/after state.
- ✓ Together, they provide full visibility into changes.



AUDIT INVESTIGATIONS

- ✓ Use CloudTrail to trace API activity and user actions.
- ✓ Use AWS Config to review configuration history.
- ✓ Correlate events and configuration changes.
- ✓ Generate evidence for security and compliance audits.
- ✓ Export reports to S3, Athena, or 3rd party tools.



KEY BENEFITS

- ✓ Complete visibility into API activity and configuration changes.
- ✓ Improve security, governance, and operational troubleshooting.
- ✓ Support compliance frameworks (PCI DSS, ISO, SOC, etc.).
- ✓ Enable forensics and root cause analysis.
- ✓ Automate monitoring and remediation of changes.

VERIQTA SOCIAL



YouTube
@veriqta



GitHub
@veriqta



LinkedIn
@veriqta



X
@veriqta



Instagram
@veriqta



Telegram
@veriqta

4. IAM

TROUBLESHOOTING



AWS Identity and Access Management (IAM) controls who can access what in your AWS environment. Most access issues are caused by permissions, policies, roles, or trust relationships.

COMMON IAM ACCESS ISSUES



PERMISSION DENIED

User or role does not have required access.



MISSING OR INCORRECT POLICY

Policy does not allow the action or resource.



RESOURCE POLICY DENY

Resource-based policy explicitly denies access.



SCP DENY

Service Control Policy blocks the action.



ROLE OR TRUST POLICY ISSUE

Role cannot be assumed or trust is misconfigured.



INSUFFICIENT PRIVILEGES

Least privilege prevents the action.

IAM TROUBLESHOOTING FLOW

- 1 **IDENTIFY THE USER/ROLE**
Confirm the IAM user or role experiencing the issue.
- 2 **CHECK THE ERROR MESSAGE**
Note the exact action, resource, and error code.
- 3 **REVIEW IAM POLICIES**
Check identity-based policies attached to the user/role.
- 4 **CHECK RESOURCE POLICIES**
Verify the resource policy does not explicitly deny access.
- 5 **CHECK SCPs**
If using AWS Organizations, review applicable SCPs.
- 6 **CHECK ROLE & TRUST POLICIES**
Ensure the role can be assumed and trust is valid.
- 7 **TEST AND VERIFY**
Retest the action after making changes.

IAM POLICY TYPES



IDENTITY-BASED POLICIES

Attached to users, groups, or roles. Control what actions are allowed.



RESOURCE-BASED POLICIES

Attached to resources (S3, SQS, KMS, etc.). Control who can access the resource.



IAM PERMISSIONS BOUNDARIES

Set the maximum permissions an identity-based policy can grant.



SERVICE CONTROL POLICIES (SCPs)

Applied at the AWS Organization level. Can allow or deny actions for accounts.



SESSION POLICIES

Passed during role assumption. Further restrict the session permissions.

KEY COMPONENTS TO CHECK



IAM POLICIES

- Review Allow and Deny statements.
- Check Action, Resource, and Condition.



RESOURCE POLICIES

- Look for explicit Deny.
- Ensure correct Principal, Action, and Resource.



SCPs

- Check if the SCP denies the action.
- Use the effective permissions view.



ROLES

- Confirm the correct role is used.
- Verify the trust relationship.



TRUST POLICIES

- Ensure the trusted entity is correct.
- Check ExternalId or MFA conditions.



ACCESS ANALYZER

- Identify external access risks.
- Analyze unused access and findings.

TROUBLESHOOTING TIPS

- ✓ Use IAM Policy Simulator to test permissions.
- ✓ Check CloudTrail for the denied action.
- ✓ Look for explicit Deny – it overrides all Allows.
- ✓ Use Access Advisor to find unused permissions.
- ✓ Follow the principle of least privilege.
- ✓ Document all IAM changes and reviews.
- ✓ Automate access reviews and alerts.
- ✓ Use Access Analyzer for continuous validation.



AWS ACCESS ANALYZER

Helps identify resources shared externally, unused access, and policy issues. Improve security and simplify auditing.



MOST IAM ISSUES ARE CAUSED BY: WRONG POLICY, WRONG RESOURCE, EXPLICIT DENY, OR BROKEN TRUST. CHECK IN THIS ORDER!

VERIQTA SOCIAL



YouTube
@veriqta



GitHub
@veriqta



LinkedIn
@veriqta



X
@veriqta



Instagram
@veriqta



Telegram
@veriqta

5. EC2 TROUBLESHOOTING



Amazon EC2 provides resizable compute capacity in the cloud. This page covers common EC2 issues and how to diagnose and resolve them quickly.

1. INSTANCE LIFECYCLE



IMPORTANT: EBS volumes persist after stop. Instance store volumes are lost when stopped/terminated.

2. BOOT FAILURES



- Instance stuck in "Pending".
- Status check failed.
- EBS volume not found or not attached.
- Invalid AMI or corrupted AMI.
- User data script failures.
- Kernel panic or OS corruption.

TROUBLESHOOTING STEPS

- 1 Check system status (AWS Health Dashboard).
- 2 Verify EBS volume and snapshot status.
- 3 Review instance console output.
- 4 Check user data logs (/var/log/cloud-init.log).
- 5 Try EC2Rescue to diagnose and fix issues.

3. SSH FAILURES

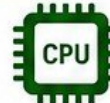


- Connection timed out.
- Incorrect key or permissions.
- Security group blocking port 22.
- OS firewall blocking SSH.
- Instance is stopped or impaired.

TROUBLESHOOTING STEPS

- 1 Verify instance status is "Running".
- 2 Check Security Group inbound rules (port 22).
- 3 Ensure correct key pair and permissions (chmod 400 key.pem).
- 4 Check OS firewall (ufw, firewall, iptables).
- 5 Use EC2 Instance Connect or Session Manager.

4. CPU SATURATION



- High CPU utilization (near 100%).
- Application process using too much CPU.
- Insufficient instance size.
- Runaway loops or spikes.
- Background processes.

TROUBLESHOOTING STEPS

- 1 Check CPU metrics in CloudWatch.
- 2 Connect and run: `top` or `htop`
- 3 Identify and stop/optimize the process.
- 4 Consider upgrading instance type.
- 5 Set up CloudWatch Alarms for CPU.

5. MEMORY ISSUES



- High memory usage / swapping.
- Out of memory (OOM) errors.
- Memory leaks in applications.
- Insufficient instance memory.
- Too many running processes.

TROUBLESHOOTING STEPS

- 1 Check memory metrics in CloudWatch.
- 2 Run `free -h` or `vmstat 1 5`.
- 3 Identify memory-hungry processes.
- 4 Optimize application or increase instance memory.
- 5 Configure swap carefully if needed.

6. DISK PROBLEMS



- Disk full (100% used).
- Inodes exhausted.
- High disk I/O wait.
- EBS volume issues.
- Filesystem corruption.

TROUBLESHOOTING STEPS

- 1 Check disk usage: `df -h`
- 2 Check inode usage: `df -i`
- 3 Check disk I/O: `iostat -x 1 5`
- 4 Clean logs, temporary files, or old data.
- 5 For EBS issues: check volume status and performance.

7. INSTANCE RECOVERY OPTIONS



EC2 INSTANCE CONNECT
Browser-based SSH access for quick troubleshooting.



SYSTEMS MANAGER SESSION MANAGER
Secure shell access without opening inbound ports.



EC2 RESCUE
Automated tool to diagnose and fix boot, network, and OS issues.



AMIS & SNAPSHOTS
Create AMIs and snapshots regularly for fast recovery and rollback.



BEST PRACTICES

- ✓ Use appropriate instance types.
- ✓ Monitor with CloudWatch.
- ✓ Keep AMIs and snapshots updated.
- ✓ Enable detailed status checks.
- ✓ Automate alerts and remediation.
- ✓ Use least privilege IAM roles.
- ✓ Document runbooks.
- ✓ Test recovery procedures.
- ✓ Review and optimize regularly.

VERIQT SOCIAL



YouTube
@veriqta



GitHub
@veriqta



LinkedIn
@veriqta



X
@veriqta



Instagram
@veriqta



Telegram
@veriqta

6. EBS TROUBLESHOOTING



Amazon Elastic Block Store (EBS) provides persistent block storage for EC2 instances. This page covers common EBS issues and how to diagnose and resolve them.



1. VOLUME ATTACHMENT ISSUES

- Volume stuck in "attaching" or "in-use".
- Volume already attached to another instance.
- Incorrect AZ or instance stop state.
- Exceeded the attachment limit.

TROUBLESHOOTING STEPS

- 1 Ensure the instance is in the same AZ.
- 2 Stop the instance before attaching.
- 3 Check if the volume is attached elsewhere.
- 4 Detach from the previous instance (if any).
- 5 Verify attachment limit (max 28 per instance).
- 6 Check EBS status: AWS Health Dashboard.



2. IOPS BOTTLENECKS

- High I/O wait and low application performance.
- Burstable volumes exceeded baseline.
- Provisioned IOPS limit reached.
- Noisy neighbor on the same host.

TROUBLESHOOTING STEPS

- 1 Check CloudWatch metric: VolumeQueueLength.
- 2 Check CloudWatch metric: VolumeRead/WriteOps.
- 3 Compare current IOPS with volume IOPS limit.
- 4 Consider increasing IOPS or using io1/io2 volumes.
- 5 Distribute volumes across instances if needed.
- 6 Optimize application I/O patterns.



3. THROUGHPUT LIMITS

- High latency with low IOPS.
- Throughput limit reached.
- Large sequential workloads affected.
- gp2/gp3 baseline limits hit.

TROUBLESHOOTING STEPS

- 1 Check CloudWatch metric: VolumeThroughputPercentage.
- 2 Verify volume type and throughput limits.
- 3 For gp2/gp3, ensure baseline throughput is sufficient.
- 4 Upgrade to gp3 and increase throughput.
- 5 For io1/io2, increase size (for higher throughput).
- 6 Optimize workload to use parallel I/O.



4. SNAPSHOT FAILURES

- Snapshot creation stuck or failed.
- Insufficient permissions.
- KMS key issues.
- Insufficient storage quota.

TROUBLESHOOTING STEPS

- 1 Check snapshot status in the EBS console.
- 2 Verify IAM permissions for CreateSnapshot.
- 3 Check KMS key permissions and status.
- 4 Ensure enough EBS snapshot storage quota.
- 5 Retry snapshot or create manual snapshot.
- 6 Check CloudTrail for error details.



5. ENCRYPTION ISSUES

- Access denied on encrypted volume.
- KMS key disabled or pending deletion.
- Incorrect key policy.
- Cross-account encryption issues.

TROUBLESHOOTING STEPS

- 1 Verify the KMS key is enabled.
- 2 Check key policy allows EBS service.
- 3 Ensure the IAM role has kms:Decrypt.
- 4 For snapshots, ensure the key is same or shared.
- 5 Recrypt or copy snapshot to a new key if needed.
- 6 Review CloudTrail for KMS errors.



6. FILESYSTEM CORRUPTION

- Unexpected shutdown or power loss.
- Kernel panic or hardware issues.
- Improper unmount or detach.
- Disk full or metadata corruption.

TROUBLESHOOTING STEPS

- 1 Check system logs (dmesg, /var/log/messages).
- 2 Run filesystem check (fsck) on unmounted volume.
- 3 Mount volume in rescue mode if needed.
- 4 Restore from snapshot if corruption is severe.
- 5 Enable regular backups and snapshots.
- 6 Investigate underlying root cause.

7. PERFORMANCE OPTIMIZATION BEST PRACTICES

Choose the Right Volume Type
Use gp3 for most workloads. Use io2 for mission-critical performance.

Right-Size Volumes
Provision enough IOPS and throughput based on workload requirements.

Monitor Continuously
Track key metrics: IOPS, throughput, latency, and queue length.

Use Snapshots Regularly
Enable automated snapshots for backup and DR.

Distribute I/O
Use multiple volumes and RAID/LVM to improve performance and availability.

Keep Systems Updated
Keep OS, drivers, and applications updated for optimal performance.



- ✓ Verify volume status and attachment
- ✓ Check CloudWatch metrics
- ✓ Review instance and AZ



- ✓ Validate IAM and KMS permissions
- ✓ Check quotas and limits
- ✓ Review logs and error messages



- ✓ Test performance and optimize
- ✓ Take regular snapshots
- ✓ Document and monitor continuously



YouTube
@veriqta



GitHub
@veriqta



LinkedIn
@veriqta



X
@veriqta



Instagram
@veriqta



Telegram
@veriqta

VERIQTA SOCIAL

7. VPC TROUBLESHOOTING



Amazon VPC (Virtual Private Cloud) networking issues are a common cause of connectivity problems in AWS. This page covers key VPC components and how to troubleshoot them.



1. ROUTE TABLES

- Incorrect or missing routes.
- Wrong target (IGW, NAT, TGW, etc.).
- Overlapping routes.
- Asymmetric routing.

TROUBLESHOOTING STEPS

- 1 Check the subnet's associated route table.
- 2 Verify the destination route exists.
- 3 Confirm the target is correct and available.
- 4 Ensure no conflicting or more specific route.
- 5 Use VPC Reachability Analyzer.



2. SECURITY GROUPS

- Inbound rules blocking traffic.
- Outbound rules too restrictive.
- Wrong protocol, port or IP.
- Instance attached to wrong SG.

TROUBLESHOOTING STEPS

- 1 Verify the security group attached to the resource.
- 2 Check inbound rules (source, port, protocol).
- 3 Check outbound rules (destination, port, protocol).
- 4 Ensure rules reference the correct IP or SG.
- 5 Use Reachability Analyzer to test connectivity.



3. NETWORK ACLs (NACLs)

- NACL rules are stateless.
- Missing ephemeral port rules (1024–65535).
- Implicit deny due to rule order.
- Subnet associated with wrong NACL.

TROUBLESHOOTING STEPS

- 1 Identify the NACL associated with the subnet.
- 2 Check inbound rules (allow return traffic).
- 3 Check outbound rules.
- 4 Ensure ephemeral ports are allowed.
- 5 Confirm rule numbers and order.



4. INTERNET GATEWAY (IGW)

- IGW not attached to the VPC.
- Missing route to IGW in route table.
- Subnet not public (no route to IGW).
- Firewall/Security Group blocking.

TROUBLESHOOTING STEPS

- 1 Verify IGW is attached to the VPC.
- 2 Check route table for 0.0.0.0/0 → IGW.
- 3 Confirm subnet is public and associated.
- 4 Check security group and NACL rules.
- 5 Test connectivity from the instance.



5. NAT GATEWAY

- NAT Gateway not in a public subnet.
- No route to NAT in private subnet.
- Elastic IP not associated.
- Incorrect NACL or SG rules.

TROUBLESHOOTING STEPS

- 1 Verify NAT Gateway is in a public subnet.
- 2 Check route table: 0.0.0.0/0 → NAT Gateway.
- 3 Confirm Elastic IP is allocated and attached.
- 4 Check NACL and Security Group rules.
- 5 Test internet access from private subnet.



6. VPC PEERING

- Peering not in "Active" state.
- Route tables missing peering routes.
- Overlapping CIDR blocks.
- SG/NACL blocking traffic.

TROUBLESHOOTING STEPS

- 1 Check peering connection status.
- 2 Add routes in both VPCs for each CIDR.
- 3 Ensure CIDRs do not overlap.
- 4 Check SG and NACL rules on both sides.
- 5 Test connectivity using Reachability Analyzer.



7. TRANSIT GATEWAY (TGW)

- VPC not attached to TGW.
- Missing routes in TGW route table.
- Incorrect route propagation.
- Blackhole or asymmetric routing.
- Appliance mode / MTU issues.
- Cross-account permission issues.

TROUBLESHOOTING STEPS

- 1 Verify VPC attachment is "Available".
- 2 Check TGW route table for correct routes.
- 3 Enable route propagation where needed.
- 4 Ensure no overlapping routes.
- 5 Verify appliance mode and MTU settings.
- 6 Test connectivity using Reachability Analyzer.

VPC TROUBLESHOOTING WORKFLOW



VPC BEST PRACTICES

- ✓ Use least-privilege rules in SG and NACL.
- ✓ Keep route tables clean and specific.
- ✓ Use Reachability Analyzer for testing.
- ✓ Monitor with VPC Flow Logs.
- ✓ Document your network design.
- ✓ Regularly review and audit configurations.

YouTube
@veriqtaGitHub
@veriqtaLinkedIn
@veriqta

VERIQTA SOCIAL

X
@veriqtaInstagram
@veriqtaTelegram
@veriqta

8. DNS AND ROUTE 53

TROUBLESHOOTING



Amazon Route 53 is AWS's scalable DNS service. This page covers common DNS and Route 53 issues and how to diagnose and resolve them.



1. DNS RESOLUTION ISSUES

- Domain not resolving.
- Incorrect nameservers.
- NXDOMAIN or SERVFAIL errors.
- Intermittent resolution failures.
- ISP or network blocking DNS.

TROUBLESHOOTING STEPS

- 1 Use nslookup, dig or host to test resolution.
- 2 Verify the domain uses Route 53 nameservers.
- 3 Check for typos in the domain name.
- 4 Test from multiple networks and locations.
- 5 Check Resolver query logs (if using Route 53 Resolver).



2. HOSTED ZONES

- Wrong or missing hosted zone.
- Records added to the wrong zone.
- SOA or NS records misconfigured.
- Unable to edit or delete a zone.
- Delegation issues.

TROUBLESHOOTING STEPS

- 1 Verify the domain is in the correct hosted zone.
- 2 Confirm NS records at the registrar point to Route 53.
- 3 Check SOA record and zone configuration.
- 4 Ensure the zone is not locked by DNSSEC issues.
- 5 Review delegation status in the domain registrar.



3. ALIAS RECORD ISSUES

- Alias not resolving to target.
- Wrong target type or Region.
- Target resource deleted or unavailable.
- Health check failing for alias target.
- Alias not working with apex domain.

TROUBLESHOOTING STEPS

- 1 Verify alias target exists and is in the correct Region.
- 2 Ensure the target type is supported (e.g., ALB, CloudFront).
- 3 Check the health of the target resource.
- 4 Confirm alias is created in the correct hosted zone.
- 5 Use dig to verify CNAME/alias response.



4. HEALTH CHECK ISSUES

- Health check shows unhealthy.
- Incorrect endpoint or port.
- Security group / firewall blocking.
- Timeouts or latency issues.
- Health check not associated with records.

TROUBLESHOOTING STEPS

- 1 Verify the health check endpoint is reachable.
- 2 Check port, protocol and path settings.
- 3 Confirm security groups and NACLs allow traffic.
- 4 Review CloudWatch metrics for the health check.
- 5 Ensure the health check is associated with records.



5. RESOLVER ENDPOINT ISSUES

- Inbound/Outbound endpoint not working.
- VPC not associated.
- Security group blocking DNS (port 53).
- No route to Resolver endpoints.
- DNS queries timing out.

TROUBLESHOOTING STEPS

- 1 Verify the endpoint status is "Available".
- 2 Ensure the VPC is associated with the endpoint.
- 3 Check security group allows TCP/UDP port 53.
- 4 Confirm route tables allow traffic to the endpoint.
- 5 Check Resolver query logs for errors.



6. PROPAGATION ISSUES

- DNS changes not visible.
- TTL still caching old records.
- Long propagation delays.
- Some locations see old records.
- Confusion with registry vs DNS propagation.

TROUBLESHOOTING STEPS

- 1 Understand TTL and wait for expiration.
- 2 Use dig @8.8.8.8 and dig @1.1.1.1 to test.
- 3 Check propagation with whatsmydns.net.
- 4 Verify you updated the correct record and zone.
- 5 Clear local DNS cache and retest.



7. FAILOVER ROUTING ISSUES

- Failover not working.
- Both primary and secondary unhealthy.
- Routing policy misconfigured.
- Health check misconfigured.

- Traffic not switching to secondary.
- Latency or geolocation routing incorrect.
- Record set priority not working.

TROUBLESHOOTING STEPS

- 1 Verify routing policy (Failover / Geolocation / Latency / Weighted).
- 2 Check health check status for primary resource.
- 3 Ensure failover record sets have correct priorities.
- 4 Confirm secondary resources are healthy and reachable.
- 5 Review CloudWatch metrics and health check logs.
- 6 Test from multiple locations and networks.



BEST PRACTICES

- ✓ Use meaningful TTL values.
- ✓ Monitor health checks and DNS metrics.
- ✓ Enable routing policies for high availability.
- ✓ Use Alias records for AWS resources.
- ✓ Test DNS changes before production.
- ✓ Keep hosted zones clean and organized.



YouTube
@veriqta



GitHub
@veriqta



LinkedIn
@veriqta



X
@veriqta



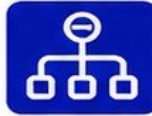
Instagram
@veriqta



Telegram
@veriqta

VERIQTA SOCIAL

9. LOAD BALANCER TROUBLESHOOTING



Elastic Load Balancing distributes incoming traffic across multiple targets. This page covers how to diagnose and resolve common issues with ALB, NLB, and their components.



1. APPLICATION LOAD BALANCER (ALB)

- 5xx errors returned to clients.
- Target not reachable.
- Incorrect listener configuration.
- HTTP to HTTPS redirection issues.
- WebSocket / HTTP/2 problems.

TROUBLESHOOTING STEPS

- 1 Check the ALB scheme (internet-facing or internal).
- 2 Verify the listener port and protocol.
- 3 Check target group health.
- 4 Review ALB access logs in S3 / CloudWatch.
- 5 Test connectivity from ALB to targets.



2. NETWORK LOAD BALANCER (NLB)

- Connection timeouts.
- Targets not receiving traffic.
- Incorrect protocol or port.
- Cross-zone issues.
- Static IP or Elastic IP issues.

TROUBLESHOOTING STEPS

- 1 Verify listener protocol (TCP / TLS / UDP).
- 2 Check target group health.
- 3 Confirm target port and security group rules.
- 4 Verify cross-zone load balancing settings.
- 5 Test from client to NLB DNS / IP.



3. TARGET GROUPS

- Targets are unhealthy.
- Wrong target type (instance / IP / Lambda).
- Targets not registering.
- Deregistration delay issues.

TROUBLESHOOTING STEPS

- 1 Check target group target type and protocol.
- 2 Verify targets are registered.
- 3 Check target health status and reason.
- 4 Confirm security groups and NACLs.
- 5 Review deregistration delay settings.



4. HEALTH CHECKS

- Targets marked unhealthy.
- Incorrect health check path / port.
- Timeout or interval too low.
- Success codes misconfigured.

TROUBLESHOOTING STEPS

- 1 Verify health check protocol, path and port.
- 2 Confirm success codes (e.g., 200-399).
- 3 Check health check timeout and interval.
- 4 Review target application logs.
- 5 Ensure security groups allow health check traffic.



5. LISTENER RULES

- Requests not matching any rule.
- Wrong priority order.
- Incorrect conditions.
- Forwarding to wrong target group.

TROUBLESHOOTING STEPS

- 1 Review listener rules and priorities.
- 2 Verify host headers, paths and conditions.
- 3 Ensure default action is correct.
- 4 Check target group associated with rule.
- 5 Use ALB access logs to confirm rule match.



6. SSL / TLS TERMINATION

- SSL handshake failures.
- Certificate expired or invalid.
- Wrong certificate on listener.
- Cipher suite mismatch.

TROUBLESHOOTING STEPS

- 1 Confirm certificate is valid and not expired.
- 2 Ensure certificate is in the correct region (ACM).
- 3 Check listener protocol and certificate association.
- 4 Test SSL using SSL Labs or OpenSSL.
- 5 Review client and server cipher compatibility.



7. STICKY SESSIONS

- Session not persisting.
- Cookie not set or lost.
- Incorrect cookie duration.

- Health check causing session drop.
- Multiple domains affecting cookie scope.

TROUBLESHOOTING STEPS

- 1 Verify stickiness is enabled on target group.
- 2 Check cookie duration and name.
- 3 Ensure clients accept cookies.
- 4 Keep listeners and rules consistent.
- 5 Check deregistration delay settings.
- 6 Check health check associated with target group.

LOAD BALANCER TROUBLESHOOTING WORKFLOW



BEST PRACTICES

- ✓ Use health checks to remove unhealthy targets.
- ✓ Keep listeners and rules simple and explicit.
- ✓ Enable access logs and monitor regularly.
- ✓ Use SSL/TLS for all client connections.
- ✓ Review target deregistration delay settings.
- ✓ Monitor metrics in CloudWatch (4xx, 5xx, Latency).


 YouTube
 @veriqta

 GitHub
 @veriqta

 LinkedIn
 @veriqta

 X
 @veriqta

 Instagram
 @veriqta

 Telegram
 @veriqta

VERIQTA SOCIAL

10. AUTO SCALING TROUBLESHOOTING



Amazon EC2 Auto Scaling automatically adjusts your capacity to maintain availability and performance.

This page covers common Auto Scaling issues and how to troubleshoot them.



1. SCALING FAILURES

- Instances fail to launch.
- Scaling activity stuck or canceled.
- Instances enter terminated state.
- Lifecycle hook failures.
- Alarm or policy misconfiguration.

TROUBLESHOOTING STEPS

- 1 Check Activity History for failure reason.
- 2 Review instance status checks.
- 3 Verify Launch Template / Launch Configuration.
- 4 Check IAM role and instance profile permissions.
- 5 Review lifecycle hook timeouts and actions.



2. LAUNCH TEMPLATES

- Invalid Launch Template version.
- Missing AMI or key pair.
- Security groups misconfigured.
- User data causing boot failure.
- IMDS or metadata access blocked.

TROUBLESHOOTING STEPS

- 1 Verify Launch Template version is correct.
- 2 Confirm AMI is valid and available.
- 3 Check security groups and subnet settings.
- 4 Review user data script for errors.
- 5 Test launching instance manually.



3. CAPACITY SHORTAGES

- Insufficient EC2 capacity in AZ.
- On-Demand capacity unavailable.
- Spot instance interruptions.
- Capacity Rebalance events.
- Max size cannot be reached.

TROUBLESHOOTING STEPS

- 1 Check EC2 service health dashboard.
- 2 Review insufficient capacity events.
- 3 Enable Capacity Rebalance for Spot fleets.
- 4 Use multiple AZs and larger subnets.
- 5 Consider On-Demand fallback or mixed instances.



4. HEALTH CHECK ISSUES

- Instances marked unhealthy.
- ELB / target group health failing.
- EC2 status checks failing.
- Grace period misconfigured.
- Application not responding.

TROUBLESHOOTING STEPS

- 1 Check target group health status.
- 2 Verify health check path, port, and protocol.
- 3 Review EC2 status checks (System & Instance).
- 4 Adjust health check grace period.
- 5 Check application logs on instances.



5. WARM POOLS

- Warm pool not maintaining instances.
- Instances not entering warm pool.
- Warm pool capacity not available.
- Lifecycle hooks delaying return.
- Stale instances in warm pool.

TROUBLESHOOTING STEPS

- 1 Verify warm pool configuration and size.
- 2 Check instance states in warm pool.
- 3 Review lifecycle hook for warm pool.
- 4 Ensure subnets and capacity are available.
- 5 Check scaling activities for warm pool events.



6. INSTANCE REFRESH

- Instance refresh stuck or failed.
- Batch replacements failing.
- Min healthy percentage too strict.
- Scale-in protection blocking refresh.
- Application errors after refresh.

TROUBLESHOOTING STEPS

- 1 Check instance refresh status and events.
- 2 Review min healthy percentage settings.
- 3 Verify instances meet health check requirements.
- 4 Disable scale-in protection if needed.
- 5 Review application and deployment changes.



7. SCALING POLICIES

- Policies not triggering.
- Incorrect metric or threshold.
- Cooldown too long.
- Predictive scaling not working.
- Step adjustments misconfigured.
- Multiple policies conflicting.
- CloudWatch alarm issues.
- Target tracking misconfigured.

TROUBLESHOOTING STEPS

- 1 Verify CloudWatch alarms are in ALARM state.
- 2 Check metric namespace, statistic, and period.
- 3 Review policy thresholds and adjustments.
- 4 Validate cooldown settings.
- 5 Check for conflicting scaling policies.
- 6 Review predictive scaling forecast and metrics.
- 7 Test scaling policy manually.



Identify the Issue



Check Activity History & Events



Analyze Root Cause



Apply Fix



Monitor & Verify

AUTO SCALING TROUBLESHOOTING WORKFLOW



BEST PRACTICES

- ✓ Use multiple AZs.
- ✓ Keep health checks accurate.
- ✓ Use warm pools for faster scaling.
- ✓ Use mixed instances.
- ✓ Monitor scaling activities.
- ✓ Test scaling policies regularly.
- ✓ Use instance refresh safely.
- ✓ Right-size your instances.
- ✓ Review alarms and metrics.



YouTube
@veriqta



GitHub
@veriqta



LinkedIn
@veriqta



X
@veriqta



Instagram
@veriqta



Telegram
@veriqta

VERIQTA SOCIAL

11. S3

TROUBLESHOOTING



Amazon S3 (Simple Storage Service) is highly durable and scalable, but issues can occur. This page covers common S3 problems and how to diagnose and resolve them.



1. ACCESS DENIED

- IAM user or role lacks permissions.
- Incorrect credential or expired key.
- Block Public Access restrictions.
- Missing permissions in bucket policy.
- Access denied through VPC endpoint.

TROUBLESHOOTING STEPS

- 1 Verify IAM user/role permissions.
- 2 Check bucket policy and ACL settings.
- 3 Review Block Public Access configuration.
- 4 Validate access through VPC endpoint policy.
- 5 Use AWS CLI with --debug for detailed error.



2. BUCKET POLICIES

- Policy syntax errors.
- Overly restrictive conditions.
- Deny statements blocking access.
- Incorrect principal or resource.
- Conflicting with IAM permissions.

TROUBLESHOOTING STEPS

- 1 Validate policy using AWS Policy Simulator.
- 2 Check for explicit Deny statements.
- 3 Verify correct Principal and Resource ARNs.
- 4 Ensure conditions (IP, MFA, etc.) are correct.
- 5 Test with temporary full-access policy.



3. VERSIONING

- Versioning not enabled.
- Unable to see previous versions.
- Accidental overwrite of objects.
- Delete markers causing confusion.
- Version suspension issues.

TROUBLESHOOTING STEPS

- 1 Check if versioning is enabled on the bucket.
- 2 List object versions using AWS CLI.
- 3 Restore previous version if needed.
- 4 Remove delete markers if required.
- 5 Understand versioning states (Enabled/Suspended).



4. REPLICATION

- Replication not configured correctly.
- IAM permissions missing for replication role.
- Objects not replicating.
- KMS encryption blocking replication.
- Replication time control issues.

TROUBLESHOOTING STEPS

- 1 Verify replication configuration and status.
- 2 Check replication IAM role permissions.
- 3 Review replication metrics and CloudWatch logs.
- 4 Ensure KMS keys are accessible in destination.
- 5 Validate destination bucket policy.



5. LIFECYCLE RULES

- Rules not applying as expected.
- Incorrect filters or prefixes.
- Objects not transitioning or expiring.
- Glacier transition failures.
- Noncurrent version rules misconfigured.

TROUBLESHOOTING STEPS

- 1 Review lifecycle rule syntax and scope.
- 2 Check rule status in Lifecycle dashboard.
- 3 Verify object tags and prefixes match rule.
- 4 Check storage class transition eligibility.
- 5 Review CloudWatch metrics for lifecycle actions.



6. PERFORMANCE

- Slow upload or download.
- High 4xx/5xx error rates.
- Throttling or request limits.
- Inefficient request patterns.
- Large object transfer issues.

TROUBLESHOOTING STEPS

- 1 Monitor S3 metrics in CloudWatch.
- 2 Check request rate limits and retries.
- 3 Use multipart upload for large files.
- 4 Optimize prefix distribution.
- 5 Enable Transfer Acceleration if needed.



7. DATA RECOVERY

- Accidentally deleted objects.
- Deleted without versioning.
- Overwritten objects.
- Corrupted uploads.
- Need to restore from backup.

TROUBLESHOOTING STEPS

- 1 Restore from object versions if available.
- 2 Check delete markers and remove if needed.
- 3 Use S3 Batch Operations for bulk restore.
- 4 Restore from backups (AWS Backup, third-party).
- 5 Enable versioning and MFA Delete for future.

S3 TROUBLESHOOTING WORKFLOW



1 Identify Issue



2 Check Permissions & Policies



3 Examine Bucket Settings



4 Review Logs & Metrics



5 Apply Fix



6 Verify & Monitor



BEST PRACTICES

- ✓ Enable Versioning on all buckets.
- ✓ Use least privilege IAM policies.
- ✓ Enable MFA Delete for protection.
- ✓ Configure lifecycle rules properly.
- ✓ Monitor with CloudWatch metrics.
- ✓ Test replication and backups regularly.
- ✓ Use meaningful bucket policies.
- ✓ Document and review access.
- ✓ Automate with IaC and tagging.



YouTube
@veriqta



GitHub
@veriqta



VERIQTA SOCIAL
LinkedIn
@veriqta



X
@veriqta



Instagram
@veriqta



Telegram
@veriqta

12. EFS

TROUBLESHOOTING



Amazon EFS (Elastic File System) provides scalable, shared file storage for Linux workloads. This page covers common EFS issues and how to diagnose and resolve them.



1. MOUNT FAILURES

- Unable to mount EFS file system.
- NFS connection timeout.
- DNS resolution issues.
- Security group or firewall blocking.
- Incorrect access point or permissions.

TROUBLESHOOTING STEPS

- 1 Verify DNS resolution for the EFS mount target.
- 2 Check security groups allow NFS (TCP 2049).
- 3 Verify mount target is in the same VPC/subnet.
- 4 Check EFS file system state (Available).
- 5 Review IAM permissions and access point settings.



2. PERFORMANCE MODES

- General Purpose mode latency issues.
- Max I/O mode not performing as expected.
- High metadata operation latency.
- Small file performance problems.
- Burst credit balance depleted.

TROUBLESHOOTING STEPS

- 1 Verify correct performance mode for workload.
- 2 Monitor CloudWatch metrics (Latency, IOPS).
- 3 Check number of clients and operations.
- 4 Optimize application for larger I/O sizes.
- 5 Consider switching to Max I/O mode if needed.



3. THROUGHPUT MODES

- Throughput limited by bursting.
- Burst credits depleted.
- Provisioned throughput too low/high.
- Unexpected throughput drops.
- Large transfers are slow.

TROUBLESHOOTING STEPS

- 1 Check current throughput mode and limits.
- 2 Monitor burst credit balance in CloudWatch.
- 3 Adjust provisioned throughput as required.
- 4 Review workload pattern and concurrency.
- 5 Use Lifecycle Management to reduce file count.



4. SECURITY ISSUES

- Access denied errors.
- IAM permissions insufficient.
- Access point permissions misconfigured.
- Cross-account access failing.
- In-transit encryption issues.

TROUBLESHOOTING STEPS

- 1 Verify IAM role and EFS policy permissions.
- 2 Check access point and POSIX user settings.
- 3 Ensure security groups allow NFS.
- 4 Verify TLS is enabled if required.
- 5 Review KMS key permissions (if using CMK).



5. AVAILABILITY ISSUES

- Mount targets unreachable.
- AZ or network connectivity problems.
- High NFS timeouts.
- Regional service issues.
- Failover taking too long.

TROUBLESHOOTING STEPS

- 1 Verify mount targets in multiple AZs.
- 2 Check subnet routing and NACLs.
- 3 Monitor network connectivity (ping, traceroute).
- 4 Review VPC endpoints and DNS settings.
- 5 Check AWS Health Dashboard for EFS events.



6. BACKUP ISSUES

- AWS Backup job failures.
- Backup not running as expected.
- Lifecycle policy not triggering.
- Backup storage limit reached.
- Restore points missing.

TROUBLESHOOTING STEPS

- 1 Verify backup plan and resource assignments.
- 2 Check AWS Backup job status and logs.
- 3 Ensure IAM role for backup has permissions.
- 4 Review lifecycle policy and retention rules.
- 5 Confirm backup vault and KMS key access.



7. RECOVERY ISSUES

- Restoring takes too long.
- Restored data incomplete.
- Mount after restore fails.
- Data inconsistency after recovery.
- Cross-region recovery problems.

TROUBLESHOOTING STEPS

- 1 Verify restore point and time.
- 2 Check restore job status and errors.
- 3 Ensure mount targets exist in target AZs.
- 4 Validate permissions and security groups.
- 5 Run file system integrity checks after restore.

EFS TROUBLESHOOTING WORKFLOW



1 Identify the Issue



2 Gather Logs & Metrics



3 Analyze Root Cause



4 Apply Fix or Configuration



5 Verify & Monitor



BEST PRACTICES

- ✓ Use access points for fine-grained permissions.
- ✓ Deploy mount targets in every AZ in use.
- ✓ Monitor EFS metrics in CloudWatch.
- ✓ Enable backups and lifecycle management.
- ✓ Use TLS for secure NFS connections.
- ✓ Optimize for large files and concurrent access.
- ✓ Review and test recovery procedures regularly.
- ✓ Keep file system and client software updated.



YouTube
@veriqta



GitHub
@veriqta



LinkedIn
@veriqta



X
@veriqta



Instagram
@veriqta



Telegram
@veriqta

VERIQTA SOCIAL

VERIQTA

13. RDS

TROUBLESHOOTING



Amazon RDS (Relational Database Service) makes it easy to set up, operate, and scale relational databases in the cloud. This page covers common RDS issues and how to diagnose and resolve them.



1. CONNECTION FAILURES

- Unable to connect to the database.
- Security group or firewall blocking.
- Incorrect endpoint or port.
- Database not accepting connections.
- Maximum connections reached.

TROUBLESHOOTING STEPS

- 1 Verify endpoint, port, username, and password.
- 2 Check security group inbound rules.
- 3 Ensure DB instance status is "available".
- 4 Check if the database is in the correct VPC/Subnet.
- 5 Review database logs for connection errors.



2. HIGH CPU

- CPU consistently high.
- Slow queries or inefficient queries.
- Missing indexes.
- Long running transactions.
- Too many connections.

TROUBLESHOOTING STEPS

- 1 Check CPU utilization in CloudWatch.
- 2 Use Performance Insights to find top waits.
- 3 Identify slow queries and optimize.
- 4 Add missing indexes where appropriate.
- 5 Review application connection pooling.



3. STORAGE FULL

- Free storage space is low.
- Autogrowth disabled or reached limit.
- Large tables or unclean data.
- Transaction log growing (for SQL Server).
- Snapshots using large storage.

TROUBLESHOOTING STEPS

- 1 Check free storage in CloudWatch.
- 2 Enable storage autoscaling if needed.
- 3 Identify large tables with unused data.
- 4 Archive or delete old data.
- 5 Clean up old snapshots and logs.



4. REPLICATION ISSUES

- Read replica lag is high.
- Replica not syncing.
- Network or latency issues.
- Binlog / WAL issues.
- Replica stopped or in error state.

TROUBLESHOOTING STEPS

- 1 Check replica lag in CloudWatch.
- 2 Verify replica status and error messages.
- 3 Check network latency between regions/AZs.
- 4 Ensure binlog retention is sufficient.
- 5 Reboot replica if stuck and monitor.



5. BACKUP ISSUES

- Automated backups failing.
- Backup taking too long.
- Insufficient storage for backup.
- Manual snapshot fails.
- Permission issues.

TROUBLESHOOTING STEPS

- 1 Check backup status in RDS dashboard.
- 2 Review failure reason in Events.
- 3 Ensure storage is sufficient.
- 4 Check KMS permissions if using encryption.
- 5 Retry backup or take manual snapshot.



6. FAILOVER ISSUES

- Failover not triggering.
- DNS not updating.
- Application not using cluster endpoint.
- Aurora failover takes too long.
- Failover target unhealthy.

TROUBLESHOOTING STEPS

- 1 Check Multi-AZ or cluster health.
- 2 Verify failover target instance status.
- 3 Confirm application uses cluster endpoint.
- 4 Review DNS TTL settings.
- 5 Test failover manually (if safe to do so).



7. PERFORMANCE INSIGHTS

- Database load is unclear.
- Identify top waits and SQL.
- Resource bottlenecks.
- High latency queries.
- Poor I/O or locking performance.
- Memory pressure.

TROUBLESHOOTING STEPS

- 1 Enable Performance Insights.
- 2 Analyze top waits by DB load.
- 3 Identify expensive SQL statements.
- 4 Compare load over time.
- 5 Optimize queries and indexes.
- 6 Monitor improvements and iterate.



1 Identify the issue



2 Check Metrics & Logs

RDS TROUBLESHOOTING WORKFLOW



3 Analyze Root Cause



4 Apply Fix



5 Monitor & Verify



BEST PRACTICES

- ✓ Use appropriate instance classes.
- ✓ Monitor metrics and alarms.
- ✓ Keep backups enabled and tested.
- ✓ Use read replicas for read scaling.
- ✓ Optimize queries and indexes.
- ✓ Use Performance Insights regularly.
- ✓ Test failover and restore plans.
- ✓ Keep security groups tight.
- ✓ Keep engines and software updated.



YouTube
@veriqta



GitHub
@veriqta



LinkedIn
@veriqta



X
@veriqta



Instagram
@veriqta



Telegram
@veriqta

14. AURORA TROUBLESHOOTING



Amazon Aurora is a high-performance, MySQL and PostgreSQL compatible database built for the cloud. This page covers common Aurora issues and how to diagnose and resolve them.



1. CLUSTER ISSUES

- Cluster unavailable or not responding.
- Writer instance crashes or restarts.
- Connectivity between instances failing.
- Quorum loss in multi-AZ cluster.
- Cluster stuck in "modifying" state.

TROUBLESHOOTING STEPS

- 1 Check Aurora cluster status in RDS console.
- 2 Review Event log for errors and warnings.
- 3 Verify security groups and network routes.
- 4 Check instance health and CPU/Memory.
- 5 If needed, reboot writer instance.



2. READER ENDPOINTS

- Reader endpoint not responding.
- No instances behind reader endpoint.
- Slow queries on reader instances.
- Connection drops to readers.
- Incorrect routing or DNS issues.

TROUBLESHOOTING STEPS

- 1 Check reader endpoint in RDS console.
- 2 Verify healthy reader instances are available.
- 3 Review CloudWatch metrics for readers.
- 4 Check DNS resolution and TTL settings.
- 5 Recreate reader endpoint if needed.



3. FAILOVER ISSUES

- Failover taking too long or failing.
- Manual failover not triggering.
- Cluster stuck in failover.
- DNS endpoint not updating.
- Applications not reconnecting.

TROUBLESHOOTING STEPS

- 1 Check cluster status and failover history.
- 2 Verify Multi-AZ configuration.
- 3 Review Event log for failover errors.
- 4 Test failover manually (if safe).
- 5 Update application retry and timeout settings.



4. REPLICATION LAG

- Reader lag is high.
- Replica falling behind writer.
- Network latency between AZ/Region.
- Large transactions causing lag.
- Binlog/WAL apply delay.

TROUBLESHOOTING STEPS

- 1 Check replica lag in RDS console.
- 2 Review CloudWatch: ReplicaLag, CPU, IOPS.
- 3 Identify long running or blocking queries.
- 4 Optimize queries and add missing indexes.
- 5 Scale up replica or increase IOPS if needed.



5. STORAGE ISSUES

- Storage full / low free space.
- Storage autoscaling not working.
- Unable to write new data.
- Snapshots taking too long.
- Unexpected storage growth.

TROUBLESHOOTING STEPS

- 1 Check free storage in RDS console.
- 2 Verify storage autoscaling is enabled.
- 3 Identify large tables or unused data.
- 4 Delete old snapshots and logs.
- 5 Archive or clean up unnecessary data.



6. PERFORMANCE ISSUES

- High CPU utilization.
- Slow queries and poor throughput.
- High I/O wait or disk latency.
- Low connections / connection errors.
- Buffer cache hit ratio low.

TROUBLESHOOTING STEPS

- 1 Review Performance Insights dashboard.
- 2 Identify top SQL by load and latency.
- 3 Optimize queries and add indexes.
- 4 Scale instance class if needed.
- 5 Check IOPS and throughput metrics.



7. RECOVERY ISSUES

- Restore from snapshot failing.
- Point-in-time recovery not working.
- Cluster not starting after restore.
- Backup missing or outdated.
- Cross-region restore failing.
- Permission issues during restore.

TROUBLESHOOTING STEPS

- 1 Verify snapshot availability and status.
- 2 Check IAM permissions for restore.
- 3 Review Event log for restore errors.
- 4 Ensure subnet, SG, and KMS access.
- 5 Try restoring to a new cluster.



1 Identify the Issue



2 Check Metrics & Logs

AURORA TROUBLESHOOTING WORKFLOW



3 Analyze Root Cause



4 Apply Fix



5 Monitor & Verify



BEST PRACTICES

- ✓ Enable Performance Insights.
- ✓ Use Multi-AZ for high availability.
- ✓ Monitor metrics and set alarms.
- ✓ Optimize queries and indexes.
- ✓ Keep backups and snapshots tested.
- ✓ Use appropriate instance classes.
- ✓ Review Event logs regularly.
- ✓ Test failover and recovery plans.
- ✓ Keep security groups minimal.



YouTube
@veriqta



GitHub
@veriqta



LinkedIn
@veriqta



X
@veriqta



Instagram
@veriqta



Telegram
@veriqta

15. DYNAMODB TROUBLESHOOTING



Amazon DynamoDB is a fully managed NoSQL database that delivers single-digit millisecond performance at any scale. This page covers the most common DynamoDB issues and how to diagnose and resolve them.



1. THROTTLING (PROVISIONED)

- Provisioned throughput exceeded.
- Too many read or write requests.
- Burst capacity exhausted.
- ThrottlingException errors.

TROUBLESHOOTING STEPS

- 1 Check CloudWatch: ThrottledRequests.
- 2 Identify table or index being throttled.
- 3 Increase RCU/WCU or enable auto scaling.
- 4 Optimize queries and reduce item sizes.
- 5 Use DAX for read-heavy workloads.



2. CAPACITY MODES

- Incorrect mode for workload.
- On-demand cost unexpectedly high.
- Auto scaling not configured.
- Sudden traffic spikes.

TROUBLESHOOTING STEPS

- 1 Choose the right mode: On-demand or Provisioned.
- 2 Review usage patterns and peak traffic.
- 3 Configure auto scaling for predictable workloads.
- 4 Monitor consumed capacity in CloudWatch.
- 5 Adjust targets and scaling policies.



3. HOT PARTITIONS

- Uneven traffic on partition key.
- Sudden spikes on a single key.
- High throttle on specific partition.
- Poor key design.

TROUBLESHOOTING STEPS

- 1 Review partition key distribution.
- 2 Use CloudWatch: SuccessfulRequestLatency.
- 3 Identify hot keys with Contributor Insights.
- 4 Redesign key or add randomness (suffix/shard).
- 5 Implement write sharding if needed.



4. INDEXES (GSI / LSI)

- Index not updating as expected.
- High throttling on index.
- Backfilling new index takes long.
- Query returning incomplete data.

TROUBLESHOOTING STEPS

- 1 Check index status and backfill progress.
- 2 Monitor index-specific throttled requests.
- 3 Ensure queries use correct key conditions.
- 4 Verify projection type and data size.
- 5 Rebuild index if data is inconsistent.



5. STREAMS

- Stream not enabled.
- Records not appearing.
- Consumer falling behind.
- Lambda/Consumer errors.

TROUBLESHOOTING STEPS

- 1 Verify Streams is ENABLED.
- 2 Check shard iterator age in CloudWatch.
- 3 Review consumer / Lambda error logs.
- 4 Increase batch size and retry settings.
- 5 Reset or recreate consumer if stuck.



6. BACKUPS

- Backup not created.
- Restore taking too long.
- PITR not enabled.
- Backup storage limits.

TROUBLESHOOTING STEPS

- 1 Verify on-demand backup status.
- 2 Enable Point-in-Time Recovery (PITR).
- 3 Check backup window and retention.
- 4 Monitor restore progress in CloudWatch.
- 5 Ensure IAM permissions for backups.



7. PERFORMANCE

- High latency.
- Slow queries or scans.
- Large items or high payload.
- Inefficient access patterns.

TROUBLESHOOTING STEPS

- 1 Review latency and throttles in CloudWatch.
- 2 Use Query instead of Scan.
- 3 Optimize key schema and access patterns.
- 4 Reduce item size and attribute count.
- 5 Enable DAX for caching when appropriate.



1 Identify Issue



2 Check CloudWatch Metrics



3 Find Root Cause



4 Apply Fix



5 Monitor & Verify

DYNAMODB TROUBLESHOOTING WORKFLOW



BEST PRACTICES

- ✓ Design for even partition distribution.
- ✓ Use auto scaling for predictable traffic.
- ✓ Monitor CloudWatch metrics and alarms.
- ✓ Keep item size small and queries specific.
- ✓ Use DAX for read-heavy workloads.
- ✓ Enable PITR and regular backups.
- ✓ Test failover and recovery regularly.
- ✓ Use IAM least privilege access.
- ✓ Keep streams consumers healthy.

VERIQTA SOCIAL



YouTube
@veriqta



GitHub
@veriqta



LinkedIn
@veriqta



X
@veriqta



Instagram
@veriqta



Telegram
@veriqta

16. LAMBDA TROUBLESHOOTING



AWS Lambda lets you run code without managing servers. This page covers common Lambda issues and how to diagnose and fix them quickly.



1. COLD STARTS

- High latency on first invocation.
- Large package or dependencies.
- VPC ENI initialization delay.
- Insufficient memory allocation.

TROUBLESHOOTING STEPS

- 1 Increase memory to improve CPU and init time.
- 2 Use provisioned concurrency for critical functions.
- 3 Optimize package size and dependencies.
- 4 Keep function outside VPC if possible.
- 5 Monitor Init Duration in CloudWatch.



2. TIMEOUTS

- Function runs longer than timeout value.
- Upstream service is slow.
- Infinite loops or large payloads.
- Database or API delays.

TROUBLESHOOTING STEPS

- 1 Check timeout setting vs actual execution time.
- 2 Review code for long loops or blocking calls.
- 3 Add retries with exponential backoff.
- 4 Optimize downstream calls and queries.
- 5 Increase memory to get more CPU if needed.



3. MEMORY LIMITS

- Function crashes with OOM (Out of Memory).
- Large payloads or data in memory.
- Memory leak in code or libraries.
- Insufficient memory allocation.

TROUBLESHOOTING STEPS

- 1 Check CloudWatch logs for OOM errors.
- 2 Increase memory allocation.
- 3 Optimize code and release unused objects.
- 4 Stream data instead of loading all at once.
- 5 Use /tmp storage for large temporary files.



4. CONCURRENCY ISSUES

- Too many concurrent executions.
- Account concurrency limit reached.
- Throttling errors (429).
- Reserved concurrency misconfigured.

TROUBLESHOOTING STEPS

- 1 Check throttles and concurrency in CloudWatch.
- 2 Increase account concurrency limit if needed.
- 3 Set reserved concurrency for critical functions.
- 4 Use SQS, Kinesis, or EventBridge to buffer traffic.
- 5 Optimize function duration and performance.



5. IAM FAILURES

- AccessDeniedException errors.
- Missing permissions for AWS services.
- Incorrect resource ARNs.
- Assume role or trust policy issues.

TROUBLESHOOTING STEPS

- 1 Review function execution role permissions.
- 2 Check resource-based policies (S3, SNS, etc.).
- 3 Verify KMS permissions if using encryption.
- 4 Test with least-privilege policy and adjust.
- 5 Check CloudTrail for denied actions.



6. EVENT FAILURES

- Event source not triggering.
- Malformed event or schema mismatch.
- Batch item failures.
- DLQ not configured.

TROUBLESHOOTING STEPS

- 1 Check event source mapping status.
- 2 Review failed records in CloudWatch logs.
- 3 Enable Dead Letter Queue (DLQ).
- 4 Validate event structure and schema.
- 5 Use retries and bisect batch on failure.



7. MONITORING & VISIBILITY

- No logs or metrics.
- Missing or incorrect log group.
- Insufficient alarms.
- Poor performance visibility.
- Lack of tracing.
- No insights on errors.

TROUBLESHOOTING STEPS

- 1 Ensure CloudWatch Logs are enabled.
- 2 Create alarms for errors, duration, and throttles.
- 3 Enable AWS X-Ray for distributed tracing.
- 4 Use Lambda Insights for performance metrics.
- 5 Build dashboards for key metrics and alerts.



1 Identify the Issue



2 Check Logs & Metrics



3 Identify Root Cause



4 Apply Fix & Optimize



5 Monitor & Verify

LAMBDA TROUBLESHOOTING WORKFLOW



BEST PRACTICES

- ✓ Keep functions small and focused.
- ✓ Optimize memory for better performance.
- ✓ Reuse connections and resources.

- ✓ Use appropriate timeouts.
- ✓ Set alarms for errors and throttles.
- ✓ Enable DLQ for all async sources.

- ✓ Follow least-privilege IAM.
- ✓ Monitor, test, and improve continuously.
- ✓ Document and version your code.

VERIQTA SOCIAL



YouTube
@veriqta



GitHub
@veriqta



LinkedIn
@veriqta



X
@veriqta



Instagram
@veriqta



Telegram
@veriqta

17. API GATEWAY TROUBLESHOOTING



Amazon API Gateway helps you create, publish, maintain, monitor, and secure APIs. This page covers common API Gateway issues and how to diagnose and resolve them.



1. 4XX ERRORS (CLIENT ERRORS)

- Bad request or invalid input.
- Unauthorized or forbidden access.
- Missing/invalid parameters or headers.
- Resource not found.

TROUBLESHOOTING STEPS

- 1 Check execution logs for 4XX details.
- 2 Validate request path, query, headers, and body.
- 3 Verify authorizer and permissions.
- 4 Check resource and method configuration.
- 5 Confirm required parameters and schemas.



2. 5XX ERRORS (SERVER ERRORS)

- Integration failure or timeout.
- Lambda errors.
- Backend service unavailable.
- Gateway internal errors.

TROUBLESHOOTING STEPS

- 1 Check execution logs for 5XX details.
- 2 Review integration response and error mapping.
- 3 Verify backend/Lambda health and timeouts.
- 4 Check IAM role permissions for integrations.
- 5 Inspect CloudWatch logs for backend errors.



3. AUTHORIZERS

- Authorizer failing or timing out.
- Incorrect permissions or token.
- Caching issues.
- Missing or invalid identity context.

TROUBLESHOOTING STEPS

- 1 Verify authorizer type and configuration.
- 2 Check authorizer logs in CloudWatch.
- 3 Validate tokens, API keys, or IAM policies.
- 4 Test authorizer endpoint directly.
- 5 Review identity source mapping.



4. INTEGRATION ISSUES

- Backend/Lambda not reachable.
- Mapping template errors.
- Timeouts or connection failures.
- Incorrect integration type.

TROUBLESHOOTING STEPS

- 1 Test integration endpoint/Lambda directly.
- 2 Check integration request and response mapping.
- 3 Verify VPC links and network connectivity.
- 4 Check timeout settings and payload size limits.
- 5 Review IAM role and permissions for integration.



5. THROTTLING

- Rate limit exceeded (429).
- Account-level or API-level throttling.
- Burst limit exceeded.
- Insufficient quotas.

TROUBLESHOOTING STEPS

- 1 Check usage plans and API key limits.
- 2 Review throttling settings (rate & burst).
- 3 Monitor API Gateway metrics in CloudWatch.
- 4 Increase limits or distribute traffic.
- 5 Implement caching or reduce request rate.



6. LOGGING & OBSERVABILITY

- Logs not enabled.
- Missing execution logs.
- Incomplete request/response data.
- Hard to trace issues.

TROUBLESHOOTING STEPS

- 1 Enable execution logging in stage settings.
- 2 Verify CloudWatch log group and IAM role.
- 3 Check detailed metrics in CloudWatch.
- 4 Use correlation IDs for tracing requests.
- 5 Create dashboards and alerts for key metrics.



7. DEPLOYMENT ISSUES

- Changes not visible after deployment.
- Stage not updated.
- Caching causing old responses.
- Incorrect stage or base path.

TROUBLESHOOTING STEPS

- 1 Verify deployment status and stage.
- 2 Clear cache and test again.
- 3 Check stage variables and settings.
- 4 Confirm base path mapping and custom domain.
- 5 Review deployment pipeline and permissions.



1 Identify the Issue



2 Check Logs & Metrics



3 Analyze Root Cause



4 Apply Fix



5 Monitor & Verify

API GATEWAY TROUBLESHOOTING WORKFLOW



BEST PRACTICES

- ✓ Validate requests before sending.
- ✓ Use meaningful error messages.
- ✓ Enable logging and detailed metrics.
- ✓ Set appropriate throttling limits.
- ✓ Use caching to reduce backend load.
- ✓ Secure APIs with least-privilege IAM.
- ✓ Test integrations independently.
- ✓ Monitor, alert, and respond quickly.
- ✓ Keep deployments versioned and tested.



YouTube
@veriqta



GitHub
@veriqta



LinkedIn
@veriqta



X
@veriqta



Instagram
@veriqta



Telegram
@veriqta

VERIQTA SOCIAL

VERIQTA

18. ECS

TROUBLESHOOTING



Amazon Elastic Container Service (ECS) makes it easy to run, stop, and manage Docker containers. This page covers common ECS issues and how to diagnose and resolve them.



1. TASK FAILURES

- Tasks stop or exit unexpectedly.
- Cannot start tasks.
- Placement failures.
- Essential container in task failed.

TROUBLESHOOTING STEPS

- 1 Check stopped task reason in ECS console.
- 2 Review task definition, CPU, memory, and limits.
- 3 Check container logs for application errors.
- 4 Verify IAM roles and permissions.
- 5 Inspect events for placement or launch failures.



2. SERVICE FAILURES

- Service not stable or not running desired count.
- Rolling updates failing.
- Tasks keep restarting.
- Deployment stuck or rolled back.

TROUBLESHOOTING STEPS

- 1 Review service events for failure reasons.
- 2 Check task definition revisions and image versions.
- 3 Verify health checks and deployment configuration.
- 4 Inspect load balancer target group health.
- 5 Check Auto Scaling and capacity provider settings.



3. CONTAINER CRASHES

- Container exits with error.
- OOMKilled (Out of Memory).
- Application crash or segmentation fault.
- Misconfiguration or missing files.

TROUBLESHOOTING STEPS

- 1 Check container logs for crash details.
- 2 Verify memory/CPU limits and reservations.
- 3 Check environment variables and secrets.
- 4 Validate application configuration.
- 5 Reproduce issue locally if needed.



4. HEALTH CHECK ISSUES

- Target marked unhealthy.
- Health check failing.
- Container not passing checks.
- Traffic not routing to tasks.

TROUBLESHOOTING STEPS

- 1 Review target group health in console.
- 2 Check health check path, port, and timeout.
- 3 Verify container and application are responding.
- 4 Check security groups and network ACLs.
- 5 Review logs for startup time and dependencies.



5. CAPACITY PROVIDERS

- Unable to place tasks.
- Insufficient capacity.
- Spot interruptions.
- Scaling not working as expected.

TROUBLESHOOTING STEPS

- 1 Check capacity provider status and events.
- 2 Verify Auto Scaling group desired capacity.
- 3 Review instance health and registration.
- 4 Check capacity provider base and weight.
- 5 Ensure tasks have correct resource requirements.



6. NETWORKING ISSUES

- Tasks cannot reach the internet.
- Cannot connect to other services.
- Port connectivity issues.
- DNS resolution failures.

TROUBLESHOOTING STEPS

- 1 Verify subnets, route tables, and NAT gateways.
- 2 Check security groups and NACL rules.
- 3 Validate VPC endpoints and service endpoints.
- 4 Test DNS resolution from within the container.
- 5 Check task ENI attachment and IP settings.



7. LOGS & OBSERVABILITY

- Logs not appearing.
- Missing or incomplete logs.
- Hard to diagnose issues.
- Need better visibility.

TROUBLESHOOTING STEPS

- 1 Confirm awslogs driver and log configuration.
- 2 Verify CloudWatch Logs group and stream.
- 3 Check container stdout/stderr output.
- 4 Use CloudWatch Logs Insights for analysis.
- 5 Enable Container Insights for metrics and alarms.



1 Identify the Issue



2 Check Events & Logs



3 Analyze Root Cause



4 Apply Fix



5 Monitor & Verify

ECS TROUBLESHOOTING WORKFLOW



BEST PRACTICES

- ✓ Use meaningful task and service names.
- ✓ Set proper resource limits and reservations.
- ✓ Enable Container Insights and CloudWatch alarms.
- ✓ Implement health checks for all services.
- ✓ Use Auto Scaling and capacity providers.
- ✓ Store logs centrally in CloudWatch Logs.
- ✓ Keep images up to date and secure.
- ✓ Automate deployments with rollback.
- ✓ Test failure scenarios regularly.



YouTube
@veriqta



GitHub
@veriqta



LinkedIn
@veriqta

VERIQTA SOCIAL

VERIQTA



X
@veriqta



Instagram
@veriqta



Telegram
@veriqta

19. EKS

TROUBLESHOOTING



Amazon EKS (Elastic Kubernetes Service) runs Kubernetes on AWS without managing the control plane. This page covers common EKS issues and how to diagnose and resolve them.



1. NODE FAILURES

- Node not Ready.
- Node terminated or unreachable.
- Kubelet or container runtime issues.
- Insufficient resources on node.

TROUBLESHOOTING STEPS

- 1 Check node status: `kubectl get nodes`
- 2 Describe node: `kubectl describe node <node>`
- 3 Check kubelet and system logs on node.
- 4 Verify node IAM role and permissions.
- 5 Replace or recycle unhealthy nodes.



2. POD FAILURES

- Pods stuck in Pending or CrashLoopBackOff.
- Container exits with error.
- Image pull failures.
- Readiness/Liveness probe failures.

TROUBLESHOOTING STEPS

- 1 Describe pod: `kubectl describe pod <pod> -n <ns>`
- 2 Check pod logs: `kubectl logs <pod> -n <ns>`
- 3 Check events: `kubectl get events -n <ns> --sort-by=.lastTimestamp`
- 4 Verify container image and registry access.
- 5 Review resource requests, limits, and probes.



3. SCHEDULING ISSUES

- Pods stuck in Pending state.
- Insufficient CPU/MEM.
- Node selectors or taints/tolerations.
- PVC or affinity/anti-affinity problems.

TROUBLESHOOTING STEPS

- 1 Describe pod and check events.
- 2 Check node resources: `kubectl top nodes`
- 3 Verify taints and tolerations.
- 4 Check node selectors and affinity rules.
- 5 Ensure PVCs are bound and available.



4. NETWORKING ISSUES

- Pods cannot reach other pods/services.
- DNS resolution failures.
- CNI plugin issues.
- NetworkPolicy blocking traffic.

TROUBLESHOOTING STEPS

- 1 Test pod-to-pod connectivity (ping, curl).
- 2 Check CoreDNS pods and logs.
- 3 Verify CNI plugin pods are running.
- 4 Review NetworkPolicies in the namespace.
- 5 Check VPC routes, security groups, and NACLs.



5. INGRESS ISSUES

- Ingress not reachable.
- 4xx/5xx from ingress controller.
- TLS/SSL certificate errors.
- Incorrect host/path routing.

TROUBLESHOOTING STEPS

- 1 Check ingress resource: `kubectl get ingress -A`
- 2 Describe ingress and controller logs.
- 3 Verify DNS records and endpoint health.
- 4 Check TLS secret and certificate validity.
- 5 Test backend service/target reachability.



6. STORAGE ISSUES

- PVC stuck in Pending.
- Volume mount failures.
- StorageClass misconfiguration.
- EBS/EFS/FSx connectivity issues.

TROUBLESHOOTING STEPS

- 1 Describe PVC and check events.
- 2 Verify StorageClass and provisioner.
- 3 Check CSI driver pods and logs.
- 4 Ensure node has access to storage (security groups).
- 5 Test volume mount on a debug pod.



7. CLUSTER RECOVERY

- Control plane unreachable.
- etcd or API server issues.
- Upgrade or add-on failures.
- Disaster recovery and restore.

TROUBLESHOOTING STEPS

- 1 Check EKS control plane status in AWS Console.
- 2 Verify IAM roles, OIDC provider, and authentication.
- 3 Review cluster and add-on logs.
- 4 Restore from Velero backup if needed.
- 5 Recreate or rebuild cluster if unrecoverable.



1 Identify the Issue



2 Gather Logs & Events



3 Analyze Root Cause



4 Apply Fix



5 Verify & Monitor

EKS TROUBLESHOOTING WORKFLOW



BEST PRACTICES

- ✓ Keep cluster and add-ons up to date.
- ✓ Use managed node groups.
- ✓ Monitor with CloudWatch and Container Insights.
- ✓ Enable cluster logging (API, Audit, Authenticator).
- ✓ Set resource requests and limits.
- ✓ Back up cluster resources and etcd data.
- ✓ Use health checks and auto healing.
- ✓ Test DR and recovery regularly.
- ✓ Follow least-privilege IAM policies.



YouTube
@veriqta



GitHub
@veriqta



LinkedIn
@veriqta

VERIQTA SOCIAL



X
@veriqta



Instagram
@veriqta



Telegram
@veriqta

20. DOCKER

TROUBLESHOOTING ON AWS



Docker runs your containers consistently across AWS services such as EC2, ECS, EKS, and Lambda. This page covers common Docker issues and how to diagnose and resolve them.



1. IMAGE FAILURES

- Image pull fails.
- Image not found.
- Authentication errors.
- Corrupted or incomplete images.

TROUBLESHOOTING STEPS

- 1 Check image name and tag.
- 2 Verify registry access and credentials.
- 3 Run: `docker pull <image:tag>` manually.
- 4 Check ECR permissions (IAM policy).
- 5 Review ECR repository and image lifecycle.



2. CONTAINER CRASHES

- Container exits unexpectedly.
- OOMKilled (Out of Memory).
- Application crashes.
- Misconfiguration or missing files.

TROUBLESHOOTING STEPS

- 1 Check exit code: `docker inspect <container>`.
- 2 Check logs: `docker logs <container>`.
- 3 Verify memory limits and reservations.
- 4 Check application configuration and dependencies.
- 5 Run container locally to reproduce the issue.



3. LOGGING ISSUES

- Logs not appearing.
- Logs incomplete or missing.
- CloudWatch Logs not receiving logs.
- Logging driver misconfiguration.

TROUBLESHOOTING STEPS

- 1 Verify logging driver configuration.
- 2 Check CloudWatch Logs group and stream.
- 3 Ensure IAM permissions for CloudWatch Logs.
- 4 Restart logging driver or container.
- 5 Check available disk space on host.



4. NETWORKING ISSUES

- Container cannot reach internet.
- Cannot connect to other containers.
- Port mapping not working.
- DNS resolution failures.

TROUBLESHOOTING STEPS

- 1 Check container network mode and settings.
- 2 Verify security groups and NACL rules.
- 3 Test connectivity from container (ping, curl, nslookup).
- 4 Check port mapping: `docker ps`.
- 5 Verify DNS settings and VPC resolver.



5. VOLUME ISSUES

- Volume not mounting.
- Permission denied.
- Data not persisting.
- Disk full on volume.

TROUBLESHOOTING STEPS

- 1 Verify volume or bind mount path.
- 2 Check file system permissions.
- 3 Inspect volume: `docker volume inspect <name>`.
- 4 Check available disk space on host.
- 5 Ensure EBS/EFS permissions (if using AWS storage).



6. SECURITY ISSUES

- Permission denied errors.
- Root user running containers.
- Vulnerabilities in images.
- Secrets exposed in images or logs.

TROUBLESHOOTING STEPS

- 1 Run containers as non-root user.
- 2 Scan images: `amazon ecr scan` or `trivy`.
- 3 Use least-privilege IAM roles for ECS/EKS.
- 4 Manage secrets using AWS Secrets Manager.
- 5 Avoid sensitive data in Dockerfile or env vars.



7. PERFORMANCE ISSUES

- High CPU usage.
- High memory usage.
- Slow I/O or disk pressure.

TROUBLESHOOTING STEPS

- 1 Monitor with CloudWatch Container Insights.
- 2 Check CPU/memory limits and reservations.
- 3 Inspect disk I/O and optimize Docker storage driver.
- 4 Review application performance and logs.
- 5 Scale resources or optimize container configuration.



1 Identify the Issue



2 Collect Logs & Info

DOCKER TROUBLESHOOTING WORKFLOW



3 Analyze Root Cause



4 Apply Fix



5 Verify & Monitor



BEST PRACTICES

- ✓ Use minimal and trusted base images.
- ✓ Keep images updated and scan regularly.
- ✓ Use `.dockerignore` to reduce image size.

- ✓ Run containers as non-root.
- ✓ Set resource limits and reservations.
- ✓ Centralize logs with CloudWatch.

- ✓ Use health checks in Dockerfile or compose.
- ✓ Manage secrets securely.
- ✓ Monitor performance continuously.

VERIQTA SOCIAL



YouTube
@veriqta



GitHub
@veriqta



LinkedIn
@veriqta



X
@veriqta



Instagram
@veriqta



Telegram
@veriqta

21. CLOUDFORMATION TROUBLESHOOTING



AWS CloudFormation helps you model and provision AWS resources as code. This page covers common CloudFormation issues and how to diagnose and resolve them.



1. STACK FAILURES

- Stack creation or update fails.
- Resource creation fails.
- Parameter or template errors.
- Insufficient permissions.

TROUBLESHOOTING STEPS

- 1 Check stack events for the failure reason.
- 2 Review the failed resource and error message.
- 3 Validate template syntax and parameters.
- 4 Verify IAM permissions and service roles.
- 5 Fix the issue and retry the operation.



2. ROLLBACKS

- Stack enters ROLLBACK_COMPLETE.
- Automatic rollback on failure.
- Resources left in inconsistent state.
- Manual rollback triggers.

TROUBLESHOOTING STEPS

- 1 Review events to understand the rollback trigger.
- 2 Identify the resource that caused the failure.
- 3 Fix the template or resource configuration.
- 4 Use ContinueUpdateRollback if stack is stuck.
- 5 Verify stack status and re-run the operation.



3. DRIFT DETECTION

- Resources modified outside of CloudFormation.
- Configuration drift detected.
- Stack no longer matches template.
- Unintentional changes.

TROUBLESHOOTING STEPS

- 1 Run drift detection on the stack.
- 2 Review drifted resources and differences.
- 3 Determine if changes are intentional.
- 4 Update template to match or revert changes.
- 5 Re-run drift detection to confirm.



4. NESTED STACKS

- Nested stack creation fails.
- Update fails due to nested stack error.
- Parameters not passed correctly.
- Dependency between stacks.

TROUBLESHOOTING STEPS

- 1 Check events for both parent and nested stack.
- 2 Validate parameters passed to nested stacks.
- 3 Ensure nested stack template is valid.
- 4 Verify IAM permissions for nested stacks.
- 5 Update nested stack first if needed.



5. CHANGE SETS

- Change set creation fails.
- Unexpected changes in change set.
- Resources replaced unexpectedly.
- Permissions or template issues.

TROUBLESHOOTING STEPS

- 1 Review change set status and details.
- 2 Validate template and parameters.
- 3 Check for unsupported resource changes.
- 4 Resolve permission issues.
- 5 Create a new change set and review carefully.



6. DEPENDENCIES

- Resources depend on other resources.
- Deletion fails due to dependencies.
- Circular dependencies.
- Implicit dependency issues.

TROUBLESHOOTING STEPS

- 1 Check error message for dependent resource.
- 2 Identify and remove dependencies manually.
- 3 Delete or update dependent resources first.
- 4 Break circular dependencies in template.
- 5 Re-run stack operation after fixing dependencies.



7. RECOVERY & REMEDIATION

- Stack stuck in UPDATE_ROLLBACK_FAILED.
- Resources cannot be deleted.
- Recovering from failed updates.
- Data or resources need protection.

TROUBLESHOOTING STEPS

- 1 Use ContinueUpdateRollback to recover stack.
- 2 Manually fix or delete failed resources if needed.
- 3 Use Retain policy to protect critical resources.
- 4 Import existing resources if they were recreated.
- 5 Validate stack is healthy and complete operation.



1 Identify Issue



2 Check Stack Events



3 Validate Template & Parameters



4 Apply Fix



5 Verify & Redeploy

CLOUDFORMATION TROUBLESHOOTING WORKFLOW



BEST PRACTICES

- ✓ Always review stack events.
- ✓ Use change sets before updates.
- ✓ Enable rollback on failure.
- ✓ Use stack policies to protect critical resources.
- ✓ Run drift detection regularly.
- ✓ Follow least-privilege IAM principles.
- ✓ Modularize with nested stacks.
- ✓ Test in non-production first.
- ✓ Document dependencies clearly.



YouTube
@veriqta



GitHub
@veriqta



LinkedIn
@veriqta



X
@veriqta



Instagram
@veriqta



Telegram
@veriqta

VERIQT A SOCIAL

VERIQT A

22. TERRAFORM TROUBLESHOOTING



Terraform helps you build and manage infrastructure as code across AWS. This page covers common Terraform issues and how to diagnose and resolve them.



1. STATE CORRUPTION

- State file is missing or invalid.
- terraform plan/apply fails.
- Resources appear to be recreated.
- Inconsistent or unexpected behavior.

TROUBLESHOOTING STEPS

- 1 Create a backup of the current state (if possible).
- 2 Run: terraform state pull and inspect the output.
- 3 Validate state: terraform state list
- 4 Restore from backup or last known good state.
- 5 Re-import resources if parts of state are lost.



2. LOCKING ISSUES

- State lock already held.
- Stale lock after interruption.
- Cannot acquire lock.
- Concurrent operation not allowed.

TROUBLESHOOTING STEPS

- 1 Check who holds the lock in the backend.
- 2 For S3 + DynamoDB: check the lock item.
- 3 If lock is stale, delete the lock item manually (only if no operation is running).
- 4 Use: terraform force-unlock <LOCK_ID>
- 5 Prevent future issues with team coordination.



3. DRIFT (CONFIGURATION DRIFT)

- Resources changed outside Terraform.
- terraform plan shows unexpected changes.
- Manual updates in console/CLI.
- Out-of-band modifications.

TROUBLESHOOTING STEPS

- 1 Run: terraform plan to identify drift.
- 2 Review each resource marked for change.
- 3 Decide: adopt, ignore, or revert the change.
- 4 Use lifecycle ignore_changes when appropriate.
- 5 Enforce IaC policy to prevent future drift.



4. IMPORTS

- Resource exists but not in state.
- Import fails.
- Incorrect import ID.
- Missing dependencies during import.

TROUBLESHOOTING STEPS

- 1 Identify correct import ID for the resource.
- 2 Run: terraform import <address> <id>
- 3 Verify resource is now in the state.
- 4 Update configuration to match real resource.
- 5 Run: terraform plan to confirm no changes.



5. PROVIDER ERRORS

- Invalid credentials or permissions.
- API rate limits.
- Unsupported resource/argument.
- Provider plugin crashes.

TROUBLESHOOTING STEPS

- 1 Check provider version and update if needed.
- 2 Verify AWS credentials and IAM permissions.
- 3 Check service quotas and API throttling.
- 4 Run: terraform init -upgrade
- 5 Enable TF_LOG=DEBUG for detailed logs.



6. DESTROY FAILURES

- Resources not destroyed.
- Dependencies blocking destroy.
- Provisioner or hook failures.
- Manual deletions causing errors.

TROUBLESHOOTING STEPS

- 1 Run: terraform plan -destroy and review.
- 2 Remove or detach dependent resources first.
- 3 Use: -target to destroy blockers (use carefully).
- 4 Fix permission or policy issues.
- 5 Re-run destroy and verify all resources are gone.



7. RECOVERY

- State lost or corrupted.
- Backend unavailable.
- Accidental deletion.
- Need to restore environment.

TROUBLESHOOTING STEPS

- 1 Restore state from versioning or backup.
- 2 Reconfigure backend and run: terraform init -reconfigure
- 3 Re-import missing resources.
- 4 Validate infrastructure with: terraform plan
- 5 Apply carefully and verify system health.



1 Identify the Issue



2 Collect Logs & State Info

TERRAFORM TROUBLESHOOTING WORKFLOW



3 Analyze Root Cause



4 Apply Fix



5 Verify & Monitor



BEST PRACTICES

- ✓ Store state in S3 with DynamoDB locking.
- ✓ Enable S3 versioning for state file.
- ✓ Run plan before every apply.
- ✓ Keep Terraform and providers updated.
- ✓ Use least-privilege IAM policies.
- ✓ Use workspaces for environments.
- ✓ Document imports and manual changes.
- ✓ Monitor drift and enforce IaC governance.



YouTube
@veriqta



GitHub
@veriqta



LinkedIn
@veriqta



X
@veriqta



Instagram
@veriqta



Telegram
@veriqta

VERIQTA SOCIAL

VERIQTA

23. CI/CD PIPELINE TROUBLESHOOTING



CI/CD pipelines automate your build, test, and deployment process. This page covers common issues across AWS native services and GitHub Actions and how to diagnose and resolve them.



1. CODECOMMIT ISSUES

- Push or pull failures.
- Permission denied.
- Repository not found.
- Branch protection errors.

TROUBLESHOOTING STEPS

- 1 Verify IAM permissions for CodeCommit.
- 2 Check repository and branch existence.
- 3 Validate HTTPS/SSH credentials.
- 4 Review branch protection and policies.
- 5 Check network or proxy configuration.



2. CODEBUILD ISSUES

- Build failures.
- Environment errors.
- Dependency install failures.
- Timeouts or OOM errors.

TROUBLESHOOTING STEPS

- 1 Review build logs in detail.
- 2 Verify buildspec.yml syntax.
- 3 Check environment variables and secrets.
- 4 Validate IAM role permissions.
- 5 Increase timeout or compute resources.



3. CODEDEPLOY ISSUES

- Deployment failures.
- Agent not available.
- Lifecycle hook failures.
- Application unhealthy.

TROUBLESHOOTING STEPS

- 1 Check deployment logs and events.
- 2 Verify CodeDeploy agent status.
- 3 Review AppSpec file and hooks.
- 4 Validate target group health.
- 5 Check IAM role and permissions.



4. CODEPIPELINE ISSUES

- Pipeline execution failures.
- Stage transitions blocked.
- Artifact issues.
- Permission errors.

TROUBLESHOOTING STEPS

- 1 Review pipeline execution history.
- 2 Check service role permissions.
- 3 Validate artifact bucket access.
- 4 Verify action configuration.
- 5 Check CloudWatch logs for details.



5. GITHUB ACTIONS ISSUES

- Workflow failures.
- Runner registration issues.
- Secrets and permissions.
- Step or action errors.

TROUBLESHOOTING STEPS

- 1 Review workflow run logs.
- 2 Check workflow YAML syntax.
- 3 Verify repository and org secrets.
- 4 Validate runner availability and status.
- 5 Check permissions and branch settings.



6. ROLLBACKS

- Failed deployment rollback.
- Previous version unavailable.
- Database migration issues.
- State inconsistency.

TROUBLESHOOTING STEPS

- 1 Verify last known good version.
- 2 Check backup and artifact availability.
- 3 Validate rollback scripts and hooks.
- 4 Review database migration state.
- 5 Monitor application health after rollback.



7. DEPLOYMENT FAILURES

- Service not starting.
- Health check failures.
- Configuration errors.
- Dependency or network issues.

TROUBLESHOOTING STEPS

- 1 Review deployment logs and CloudWatch metrics.
- 2 Check target group and instance/container health.
- 3 Validate environment variables and configuration.
- 4 Test connectivity to dependencies and databases.
- 5 Rollback if necessary and fix root cause.



Identify the Issue



Check Logs & Events



Find Root Cause



Apply Fix



Verify & Monitor

CI/CD TROUBLESHOOTING WORKFLOW



BEST PRACTICES

- ✓ Keep pipelines simple and modular.
- ✓ Use versioned artifacts and IaC.
- ✓ Automate tests before deployments.
- ✓ Monitor every stage and set alerts.
- ✓ Use least-privilege IAM roles.
- ✓ Document runbooks and recovery steps.



YouTube
@veriqta



GitHub
@veriqta



LinkedIn
@veriqta



X
@veriqta



Instagram
@veriqta



Telegram
@veriqta

24. CLOUDFRONT TROUBLESHOOTING



Amazon CloudFront delivers content globally with low latency and high availability. This page covers common CloudFront issues and how to diagnose and resolve them.



1. CACHE ISSUES

- Content not updating.
- Wrong content served.
- Cache key problems.
- Unexpected TTL behavior.

TROUBLESHOOTING STEPS

- 1 Check object TTL and Cache-Control headers.
- 2 Verify cache key settings.
- 3 Inspect headers in viewer request and origin response.
- 4 Create invalidation if content changed.
- 5 Use Cache Policy and Origin Request Policy correctly.



2. SSL / TLS ISSUES

- SSL handshake failures.
- Certificate expired.
- SNI or protocol mismatch.
- Mixed content errors.

TROUBLESHOOTING STEPS

- 1 Verify ACM certificate status and domain.
- 2 Check certificate is in us-east-1.
- 3 Confirm minimum TLS version.
- 4 Review SSL/TLS negotiation in viewer policy.
- 5 Validate certificate chain with SSL Labs.



3. ORIGIN ISSUES

- 502 Bad Gateway errors.
- Connection timeouts.
- Origin not reachable.
- Incorrect origin domain.

TROUBLESHOOTING STEPS

- 1 Check origin health and availability.
- 2 Verify Security Groups and firewall.
- 3 Confirm origin domain and protocol.
- 4 Review origin response timeouts.
- 5 Test origin directly outside CloudFront.



4. WAF INTEGRATION ISSUES

- Requests blocked by WAF.
- False positives.
- Rule misconfiguration.
- IP reputation blocks.

TROUBLESHOOTING STEPS

- 1 Check WAF logs in CloudWatch.
- 2 Identify blocked rule and match conditions.
- 3 Create rule exclusions if needed.
- 4 Test request using WAF test tools.
- 5 Monitor WAF metrics and sampled requests.



5. INVALIDATIONS

- Invalidation not working.
- Old content still visible.
- Wildcard issues.
- High invalidation cost.

TROUBLESHOOTING STEPS

- 1 Verify invalidation status is Completed.
- 2 Check object paths and leading slashes.
- 3 Use versioning to avoid frequent invalidations.
- 4 Invalidate only changed files or directories.
- 5 Monitor invalidation cost in billing.



6. HEADERS ISSUES

- Missing security headers.
- Wrong content type.
- Header not forwarded.
- CORS errors.

TROUBLESHOOTING STEPS

- 1 Review viewer response headers.
- 2 Check cache and origin request policies.
- 3 Verify CORS configuration.
- 4 Ensure required headers are forwarded.
- 5 Check for header size limits.



7. PERFORMANCE ISSUES

- High latency.
- Low hit ratio.
- Slow TTFB.
- Bandwidth or throttling issues.

TROUBLESHOOTING STEPS

- 1 Check CloudFront metrics (Latencies, 5xx, Hit Ratio).
- 2 Review origin response time and size.
- 3 Optimize compression and object size.
- 4 Use appropriate cache policies.
- 5 Enable HTTP/2, HTTP/3 and keep-alive.



1 Identify the Issue



2 Check Logs & Metrics



3 Find Root Cause



4 Apply Fix



5 Verify & Monitor

CLOUDFRONT TROUBLESHOOTING WORKFLOW



BEST PRACTICES

- ✓ Use versioned file names for assets.
- ✓ Set proper Cache-Control headers.
- ✓ Enable compression (Brotli/Gzip).
- ✓ Monitor CloudWatch metrics and alarms.

- ✓ Use least-privilege IAM roles.
- ✓ Test changes in lower environments.
- ✓ Document distributions and policies.
- ✓ Review and rotate certificates regularly.



YouTube
@veriqta



GitHub
@veriqta



LinkedIn
@veriqta



X
@veriqta



Instagram
@veriqta



Telegram
@veriqta

25. AWS WAF

TROUBLESHOOTING



AWS WAF helps protect your applications from common web exploits, bots, and abusive traffic. This page covers common AWS WAF issues and how to diagnose and resolve them.



1. RULE EVALUATION ISSUES

- Rules not evaluating as expected.
- Incorrect rule priority.
- Logic or statement errors.
- Label or ARN references wrong.

TROUBLESHOOTING STEPS

- Review rule order and priorities.
- Use WAF logs to see rule matches.
- Validate rule conditions and statements.
- Check labels, ARNs, and text transforms.
- Test with AWS WAF console "Sample requests".



2. FALSE POSITIVES

- Legitimate traffic blocked.
- Overly strict conditions.
- Bad IP reputation matches.
- Missing exceptions.

TROUBLESHOOTING STEPS

- Review blocked requests in logs.
- Identify rule(s) causing the block.
- Add exclusions or scope-down statements.
- Use IP sets or allow lists when needed.
- Test changes using sample requests.



3. RATE LIMITING ISSUES

- Legitimate users throttled.
- Too aggressive rate limits.
- Incorrect aggregation key.
- Burst limits too low.

TROUBLESHOOTING STEPS

- Review rate-based rule metrics.
- Check aggregation key (IP, header, etc.).
- Increase rate limit or burst limit.
- Add exceptions for trusted traffic.
- Monitor and tune thresholds.



4. MANAGED RULE ISSUES

- Managed rule blocking legit requests.
- Rule group updates causing issues.
- Missing exclusions.
- Version conflicts.

TROUBLESHOOTING STEPS

- Check managed rule group version.
- Review AWS Managed Rules changelog.
- Examine blocked requests in logs.
- Add exclusions or scope-down rules.
- Override or disable specific rules if needed.



5. LOGGING ISSUES

- Logs not appearing in Kinesis/Firehose.
- S3 delivery failures.
- WAF logging not enabled.
- Permission errors.

TROUBLESHOOTING STEPS

- Verify logging configuration in WAF.
- Check Kinesis/Firehose/S3 permissions.
- Confirm delivery stream is healthy.
- Review CloudWatch and service logs.
- Validate S3 bucket policy and encryption.

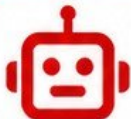


6. WEB ACL ISSUES

- Web ACL not associated.
- Changes not taking effect.
- Wrong scope (REGIONAL/CLOUDFRONT).
- Association conflicts.

TROUBLESHOOTING STEPS

- Verify Web ACL association.
- Confirm correct scope for resource.
- Check for multiple conflicting ACLs.
- Ensure propagation is complete.
- Re-associate or update Web ACL.



7. BOT PROTECTION ISSUES

- Allowed bots causing abuse.
- Blocked good bots.
- CAPTCHA/Challenge over-triggering.
- Bot Control rule misconfiguration.

TROUBLESHOOTING STEPS

- Review Bot Control and Bot rules logs.
- Analyze traffic in Bot Control dashboard.
- Adjust inspection level (Common/Targeted).
- Exclude trusted bots using labels or rules.
- Monitor metrics and fine-tune settings.



Identify
Issue



Review
WAF Logs



Analyze
Root Cause



Apply
Fix



Validate &
Monitor

AWS WAF TROUBLESHOOTING WORKFLOW



BEST PRACTICES

- Use least-privilege IAM roles.
- Enable WAF logging to CloudWatch and S3.
- Start in Count mode before blocking.
- Keep managed rule groups updated.
- Use labels to build granular rules.
- Monitor metrics and alarms continuously.



YouTube
@veriqta



GitHub
@veriqta



LinkedIn
@veriqta



X
@veriqta



Instagram
@veriqta



Telegram
@veriqta

26. AWS SECRETS MANAGER TROUBLESHOOTING



AWS Secrets Manager helps you securely store, rotate, and manage secrets. This page covers common Secrets Manager issues and how to diagnose and resolve them.



1. SECRET ROTATION ISSUES

- Rotation failing.
- Lambda errors.
- Missing permissions.
- Incorrect rotation configuration.

TROUBLESHOOTING STEPS

- 1 Review rotation Lambda logs.
- 2 Verify rotation schedule and status.
- 3 Check Lambda permissions and VPC access.
- 4 Validate secret schema and AWSCURRENT.
- 5 Run rotation test in staging first.



2. ACCESS FAILURES

- Access denied errors.
- Cannot retrieve secret.
- Decryption failures.
- Cross-account access issues.

TROUBLESHOOTING STEPS

- 1 Check IAM permissions for GetSecretValue.
- 2 Verify resource policy and conditions.
- 3 Confirm secret exists in correct Region.
- 4 Validate KMS key permissions.
- 5 Check session policies or SCP restrictions.



3. VERSION CONFLICTS

- AWSCURRENT missing.
- Stale or duplicate versions.
- Version staging label issues.
- Secret version not updating.

TROUBLESHOOTING STEPS

- 1 List secret versions and staging labels.
- 2 Ensure one version has AWSCURRENT.
- 3 Move or update staging labels correctly.
- 4 Delete orphaned versions if necessary.
- 5 Validate application is using latest version.



4. KMS INTEGRATION ISSUES

- Unable to decrypt secret.
- KMS key disabled.
- Key policy issues.
- Grants or permissions missing.

TROUBLESHOOTING STEPS

- 1 Verify KMS key is enabled and accessible.
- 2 Check key policy allows Secrets Manager.
- 3 Confirm IAM user/role has kms:Decrypt.
- 4 Review CloudTrail for KMS errors.
- 5 Test decrypt using AWS CLI.



5. IAM PERMISSIONS ISSUES

- Insufficient permissions.
- PassRole failures.
- Boundary or permission conflicts.
- Service-linked role missing.

TROUBLESHOOTING STEPS

- 1 Verify IAM policy for required actions.
- 2 Check permissions boundary restrictions.
- 3 Confirm iam:PassRole for rotation Lambda.
- 4 Ensure service-linked role exists.
- 5 Simulate policy to identify denied actions.



6. RECOVERY SCENARIOS

- Secret deleted accidentally.
- Rotation Lambda broken.
- Permission lockout.
- Region/Account recovery.

TROUBLESHOOTING STEPS

- 1 Restore secret from recovery window.
- 2 Recreate secret if recovery window expired.
- 3 Fix IAM/KMS issues causing lockout.
- 4 Rebuild rotation Lambda and test.
- 5 Validate application connectivity.



7. BEST PRACTICES

- Use customer-managed KMS keys.
- Enable automatic secret rotation.
- Follow least-privilege IAM permissions.
- Use meaningful secret naming.
- Use resource policies for cross-account access.
- Monitor secret access and rotation.
- Tag secrets for ownership and environment.
- Test rotation and recovery regularly.



Identify the Issue



Check Logs & Metrics



Analyze Root Cause



Apply Fix



Verify & Monitor

SECRETS MANAGER TROUBLESHOOTING WORKFLOW



KEY POINTS

- ✓ Check IAM, KMS, and resource policies first.
- ✓ Use CloudWatch and CloudTrail for visibility.
- ✓ Understand secret versions and staging labels.
- ✓ Test rotation and recovery frequently.
- ✓ Automate alerts for failures and anomalies.
- ✓ Secure, monitor, and document your secrets.



YouTube
@veriqta



GitHub
@veriqta



LinkedIn
@veriqta



X
@veriqta



Instagram
@veriqta



Telegram
@veriqta

27. KMS

TROUBLESHOOTING



AWS Key Management Service (KMS) secures and controls encryption keys. This page covers common KMS issues and how to diagnose and resolve them.



1. KEY POLICIES ISSUES

- Incorrect or overly restrictive policy.
- Missing principal permissions.
- Deny statements blocking access.
- Policy changes not applying.

TROUBLESHOOTING STEPS

- 1 Review key policy JSON for correctness.
- 2 Verify principal ARN and permissions.
- 3 Check for explicit Deny statements.
- 4 Use IAM Policy Simulator to test access.
- 5 Ensure policy is attached to the key.



2. ENCRYPTION FAILURES

- AccessDeniedException.
- KMS key not enabled.
- Unsupported algorithm.
- Service quota exceeded.

TROUBLESHOOTING STEPS

- 1 Verify key is enabled and in correct Region.
- 2 Check IAM permissions (kms:Encrypt).
- 3 Confirm request uses correct key ID/ARN.
- 4 Validate supported encryption algorithm.
- 5 Review CloudTrail for error details.



3. GRANTS ISSUES

- Grant not found.
- Grant revoked or expired.
- Insufficient grant permissions.
- Too many grants.

TROUBLESHOOTING STEPS

- 1 List grants using AWS CLI or console.
- 2 Verify grant is active and not revoked.
- 3 Check permissions included in grant.
- 4 Recreate grant if necessary.
- 5 Clean up unused grants.



4. KEY ROTATION ISSUES

- Automatic rotation not enabled.
- Rotation Lambda failures.
- New key version not active.
- Rotation schedule misconfigured.

TROUBLESHOOTING STEPS

- 1 Verify automatic rotation is enabled.
- 2 Check rotation Lambda logs.
- 3 Ensure key has correct rotation role.
- 4 Confirm new key version is enabled.
- 5 Run rotation test in staging.



5. CROSS-ACCOUNT ACCESS

- Access denied cross-account.
- Trust relationship missing.
- KMS key policy not allowing.
- External principal not authorized.

TROUBLESHOOTING STEPS

- 1 Verify key policy allows external account.
- 2 Check principal ARN in key policy.
- 3 Confirm external account IAM role exists.
- 4 Ensure proper permissions are granted.
- 5 Test access using assumed role.



6. DECRYPTION FAILURES

- AccessDeniedException.
- Ciphertext not from this key.
- Key disabled or scheduled deletion.
- Incorrect encryption context.

TROUBLESHOOTING STEPS

- 1 Verify kms:Decrypt permission.
- 2 Ensure ciphertext was encrypted by key.
- 3 Check key state (enabled, not deleted).
- 4 Verify encryption context matches.
- 5 Review CloudTrail for failure reason.



7. AUDITING AND MONITORING

- No CloudTrail logging.
- Missing key usage visibility.
- Unusual key usage.
- Compliance requirements.

TROUBLESHOOTING STEPS

- 1 Enable CloudTrail for KMS events.
- 2 Review key usage in CloudTrail and CloudWatch.
- 3 Use AWS Config to monitor key changes.
- 4 Set CloudWatch alarms for unusual activity.
- 5 Enable AWS Config rule: kms-key-rotation-enabled.



Identify
Issue



Check
Logs



Analyze
Root Cause



Apply
Fix



Verify &
Monitor

KMS TROUBLESHOOTING WORKFLOW



KEY POINTS

- ✓ Use least-privilege IAM and key policies.
- ✓ Enable automatic key rotation.
- ✓ Monitor KMS with CloudTrail and CloudWatch.
- ✓ Use grants for fine-grained temporary access.

- ✓ Test cross-account access before production.
- ✓ Back up and document key policies.
- ✓ Review and audit KMS keys regularly.
- ✓ Tag keys for ownership and compliance.



YouTube
@veriqta



GitHub
@veriqta



LinkedIn
@veriqta



X
@veriqta



Instagram
@veriqta



Telegram
@veriqta

28. SNS AND SQS

TROUBLESHOOTING



Amazon SNS and SQS enable reliable, scalable, and decoupled messaging. This page covers common SNS and SQS issues and how to diagnose and resolve them.



1. MESSAGE FAILURES

- Messages not reaching consumers.
- Publish or send failures.
- Large message size.
- Service or network errors.

TROUBLESHOOTING STEPS

- 1 Check publish/send response for errors.
- 2 Verify IAM permissions for SNS/SQS.
- 3 Confirm correct topic/queue and region.
- 4 Check message size limits (SNS 256 KB, SQS 256 KB standard, 1 MB FIFO).
- 5 Review CloudWatch metrics and logs.



2. DEAD-LETTER QUEUES

- Messages sent to DLQ.
- Consumer failures.
- Max receives exceeded.
- Poison messages.

TROUBLESHOOTING STEPS

- 1 Check DLQ for messages.
- 2 Inspect message and error details.
- 3 Fix root cause in consumer code.
- 4 Reset receive count or redrive message.
- 5 Reprocess after resolving the issue.



3. FIFO ISSUES

- Messages out of order.
- Deduplication failures.
- ContentBasedDeduplication issues.
- Message group blocking.

TROUBLESHOOTING STEPS

- 1 Verify MessageGroupId is set correctly.
- 2 Check DeduplicationId uniqueness.
- 3 Ensure ContentBasedDeduplication is enabled if needed.
- 4 Confirm no in-flight messages blocking the group.
- 5 Monitor FIFO metrics for throttling.



4. DELIVERY RETRIES

- Retries causing duplicates.
- Endpoint not reachable.
- Backoff and retry limits.
- Throttling or rate limits.

TROUBLESHOOTING STEPS

- 1 Review retry policy and max attempts.
- 2 Check endpoint health and availability.
- 3 Verify subscription/queue policies.
- 4 Adjust backoff or retry settings.
- 5 Use idempotent consumers.



5. VISIBILITY TIMEOUT ISSUES

- Messages become visible too soon.
- Message processed multiple times.
- Long running processing.
- Visibility not extended.

TROUBLESHOOTING STEPS

- 1 Check visibility timeout setting.
- 2 Extend visibility during processing.
- 3 Monitor ApproximateReceiveCount.
- 4 Handle failures and send to DLQ.
- 5 Align timeout with max processing time.



6. MONITORING ISSUES

- No metrics or logs.
- Missing alarms.
- High error rates.
- Lag not detected.

TROUBLESHOOTING STEPS

- 1 Verify CloudWatch metrics are enabled.
- 2 Check alarms and thresholds.
- 3 Use CloudWatch Logs for consumers.
- 4 Monitor age of oldest message.
- 5 Enable detailed error logging.



7. RECOVERY SCENARIOS

- Consumer service down.
- Queue backlog too high.
- Corrupted or invalid messages.
- Accidental deletion.

TROUBLESHOOTING STEPS

- 1 Restore consumer service and scale as needed.
- 2 Increase visibility timeout and worker capacity.
- 3 Isolate bad messages and move to DLQ.
- 4 Use redrive policy to recover messages.
- 5 Verify permissions and re-publish if required.



Identify
Issue



Check Logs &
Metrics



Analyze
Root Cause



Apply
Fix



Verify &
Monitor

SNS / SQS TROUBLESHOOTING WORKFLOW



KEY POINTS

- ✓ Use DLQs to capture and analyze failures.
- ✓ Monitor message age and queue depth.
- ✓ Design consumers to be idempotent.
- ✓ Set appropriate visibility timeout and retries.
- ✓ Use CloudWatch for metrics and alarms.
- ✓ Test failure scenarios regularly.
- ✓ Keep message payloads within limits.
- ✓ Automate recovery and alerting.



YouTube
@veriqta



GitHub
@veriqta



LinkedIn
@veriqta



X
@veriqta



Instagram
@veriqta



Telegram
@veriqta

29. EVENTBRIDGE TROUBLESHOOTING



Amazon EventBridge enables event-driven architectures at scale. This page covers common EventBridge issues and how to diagnose and resolve them.



1. RULE FAILURES

- Rules not triggering.
- Disabled or misconfigured rule.
- Event bus permissions issues.
- Service or quota errors.

TROUBLESHOOTING STEPS

- 1 Verify rule is enabled.
- 2 Check rule metrics (MatchedEvents).
- 3 Review CloudWatch Logs for errors.
- 4 Validate event bus permissions.
- 5 Check account limits and service health.



2. EVENT PATTERN ISSUES

- Pattern not matching events.
- Incorrect JSON structure.
- Case sensitivity problems.
- Missing detail fields.

TROUBLESHOOTING STEPS

- 1 Validate pattern JSON syntax.
- 2 Test pattern using "Test event pattern".
- 3 Check event structure in CloudTrail.
- 4 Ensure correct source and detail-type.
- 5 Use wildcards carefully and test again.



3. TARGET ISSUES

- Target invocation failures.
- Permissions or trust issues.
- Throttling or service errors.
- Incorrect target configuration.

TROUBLESHOOTING STEPS

- 1 Verify target IAM role permissions.
- 2 Check target health and availability.
- 3 Review CloudWatch Logs for target.
- 4 Validate input transformer settings.
- 5 Check DLQ for failed events.



4. RETRY AND DELIVERY ISSUES

- Events not delivered.
- Retry attempts exhausted.
- Max age exceeded.
- Throttling or transient errors.

TROUBLESHOOTING STEPS

- 1 Review retry policy and max attempts.
- 2 Check event age and expiration.
- 3 Verify target service quotas.
- 4 Review DLQ for failed events.
- 5 Adjust retry policy and test again.



5. DEAD-LETTER QUEUES (DLQ)

- Messages in DLQ.
- Incorrect DLQ settings.
- Permissions for DLQ.
- High volume in DLQ.

TROUBLESHOOTING STEPS

- 1 Check DLQ policy and permissions.
- 2 Inspect failed event payloads.
- 3 Identify root cause from error details.
- 4 Fix issues and re-drive messages.
- 5 Monitor DLQ metrics and alarms.



6. ARCHIVES ISSUES

- Archive not capturing events.
- Retention period problems.
- Archive permissions.
- Ingestion failures.

TROUBLESHOOTING STEPS

- 1 Verify archive is active and enabled.
- 2 Check event bus and rule association.
- 3 Validate archive IAM permissions.
- 4 Review failed entries in CloudWatch.
- 5 Confirm retention and storage limits.



7. REPLAY ISSUES

- Replay not starting.
- Events not re-delivered.
- Conflicts with active rules.
- Permission or quota errors.

TROUBLESHOOTING STEPS

- 1 Verify archive has events in time range.
- 2 Check replay IAM role permissions.
- 3 Ensure target rules are active.
- 4 Review replay status and failures.
- 5 Monitor CloudWatch metrics for replay.



Identify Issue



Check Logs & Metrics



Analyze Root Cause



Apply Fix



Verify & Monitor

EVENTBRIDGE TROUBLESHOOTING WORKFLOW



BEST PRACTICES

- ✓ Use descriptive rule and event bus names.
- ✓ Keep event patterns simple and test.
- ✓ Monitor EventBridge metrics and logs.
- ✓ Use DLQs for all critical targets.

- ✓ Archive important events for replay.
- ✓ Set appropriate retry policy and max age.
- ✓ Follow least-privilege IAM permissions.
- ✓ Test rules and targets in lower environments.



YouTube
@veriqta



GitHub
@veriqta



LinkedIn
@veriqta



X
@veriqta



Instagram
@veriqta



Telegram
@veriqta

30. STEP FUNCTIONS TROUBLESHOOTING



AWS Step Functions orchestrates workflows across AWS services. This page covers common Step Functions issues and how to diagnose and resolve them.



1. WORKFLOW FAILURES

- Execution failed.
- State machine stopped unexpectedly.
- Task, service integration, or Lambda errors.
- Permission or throttling issues.

TROUBLESHOOTING STEPS

- 1 Review execution history for error.
- 2 Check state output and error cause.
- 3 Verify IAM permissions for resources.
- 4 Review CloudWatch Logs for details.
- 5 Test the failing state or service manually.



2. STATE TRANSITION ISSUES

- Invalid state transition.
- Incorrect Next or End configuration.
- Choice state not matching.
- Parallel or Map state errors.

TROUBLESHOOTING STEPS

- 1 Validate state machine definition (ASL).
- 2 Check transitions and state names.
- 3 Review input and output paths.
- 4 Test Choice and Condition logic.
- 5 Use Step Functions Local to validate.



3. RETRY AND BACKOFF ISSUES

- Retries not working as expected.
- Max attempts reached.
- Exponential backoff misconfigured.
- Throttling or concurrency limits.

TROUBLESHOOTING STEPS

- 1 Review Retry field in state definition.
- 2 Check ErrorEquals and BackoffRate.
- 3 Verify MaxAttempts setting.
- 4 Check service quotas and throttling.
- 5 Adjust retry strategy and test again.



4. ERROR HANDLING ISSUES

- Catch blocks not triggered.
- Unhandled exceptions.
- States going to Fail state.
- Incorrect error mapping.

TROUBLESHOOTING STEPS

- 1 Check Catch blocks and error types.
- 2 Ensure ErrorEquals matches correctly.
- 3 Verify ResultPath and ResultSelector.
- 4 Check if error is retryable or terminal.
- 5 Add fallback or compensating actions.



5. TIMEOUT ISSUES

- Task timed out.
- Heartbeat timeout.
- Execution timeout.
- Wait state long running.

TROUBLESHOOTING STEPS

- 1 Review TimeoutSeconds in state.
- 2 Check HeartbeatSeconds if used.
- 3 Review overall execution timeout.
- 4 Verify downstream service latency.
- 5 Increase timeout or optimize task.



6. MONITORING AND VISIBILITY

- Missing logs or metrics.
- Difficult to trace execution.
- No alerts on failures.
- Limited execution history.

TROUBLESHOOTING STEPS

- 1 Enable CloudWatch Logs (ALL level).
- 2 Review execution metrics in CloudWatch.
- 3 Set up CloudWatch Alarms.
- 4 Use X-Ray for distributed tracing.
- 5 Export execution history if needed.



7. RECOVERY SCENARIOS

- Rerun failed execution.
- Redrive from a specific state.
- Fix and redeploy state machine.
- Handle partial or side effects.
- Use compensating transactions.

TROUBLESHOOTING STEPS

- 1 Identify failure point from execution history.
- 2 Fix code, permissions, or configuration.
- 3 Use "Start execution" with the same or new input.
- 4 For long workflows, consider redrive or resume.
- 5 Validate final output and downstream impact.



Identify Issue



Review Execution History & Logs



Analyze Root Cause



Apply Fix



Verify & Monitor



BEST PRACTICES

- ✓ Design for idempotency and retries.
- ✓ Use meaningful state and execution names.
- ✓ Capture and log errors with context.
- ✓ Apply least-privilege IAM permissions.

- ✓ Set appropriate timeouts and retries.
- ✓ Monitor executions and set alerts.
- ✓ Test workflows in lower environments.
- ✓ Use DLQ or human task for critical failures.



YouTube
@veriqta



GitHub
@veriqta



LinkedIn
@veriqta



X
@veriqta



Instagram
@veriqta



Telegram
@veriqta

31. PRODUCTION NETWORKING TROUBLESHOOTING



Reliable networking is critical in production.

This page covers end-to-end networking troubleshooting techniques and AWS tools to diagnose and resolve issues.

1. END-TO-END PACKET FLOW



- Understand the full path from source to destination.
- Identify each hop, protocol, and component.
- Know where failures can occur.
- Verify routing, security, and return path.

TROUBLESHOOTING STEPS

- 1 Map the architecture and traffic path.
- 2 Verify route tables for each subnet.
- 3 Check security groups and NACLs.
- 4 Validate return path and asymmetric routing.
- 5 Test from multiple locations and components.

2. VPC FLOW LOGS



- Capture network traffic metadata.
- Identify ACCEPT or REJECT.
- See source, destination, port, protocol, and bytes.
- Essential for security and connectivity issues.

TROUBLESHOOTING STEPS

- 1 Enable Flow Logs at VPC, subnet, or ENI level.
- 2 Send logs to CloudWatch Logs or S3.
- 3 Filter by source IP, destination IP, or port.
- 4 Analyze REJECT entries and reasons.
- 5 Correlate with timestamps and components.

3. REACHABILITY ANALYZER



- Simulate connectivity between resources.
- Identifies blocking components.
- Validates routing, SGs, NACLs, and more.
- Works across accounts and Regions.

TROUBLESHOOTING STEPS

- 1 Open Reachability Analyzer.
- 2 Choose source and destination resources.
- 3 Run analysis and review path.
- 4 Identify blocking points and reasons.
- 5 Fix configuration and re-run analysis.



4. NETWORK ACL ANALYSIS

- Stateless and subnet-level.
- Allow rules are specific.
- Implicit deny at the end.
- Ephemeral ports must be allowed.

TROUBLESHOOTING STEPS

- 1 Check inbound and outbound NACL rules.
- 2 Verify ephemeral ports (1024–65535).
- 3 Ensure return traffic is allowed.
- 4 Check rule number order and priority.
- 5 Test again and confirm connectivity.



5. SECURITY GROUP TRACING

- Stateful and instance-level.
- Allow rules are required.
- Return traffic is automatically allowed.
- Track rules impacting traffic.

TROUBLESHOOTING STEPS

- 1 Review SG inbound rules for source.
- 2 Review SG outbound rules for destination.
- 3 Check referenced SGs and dependencies.
- 4 Use VPC Flow Logs for confirmation.
- 5 Apply least-privilege and retest.



6. DNS DEBUGGING

- DNS is critical for connectivity.
- Issues cause timeouts and failures.
- Validate resolution, records, and TTL.

TROUBLESHOOTING STEPS

- 1 Use nslookup or dig to test resolution.
- 2 Verify record type, value, and TTL.
- 3 Check Route 53 health and routing.
- 4 Validate resolver configuration.
- 5 Clear caches and test again.



7. CONNECTIVITY TESTING

- Test connectivity at each layer.
- Use multiple tools and methods.
- Test from source and destination.
- Validate ports, protocols, and paths.

TROUBLESHOOTING STEPS

- 1 Ping or ICMP test between components.
- 2 Use telnet, nc, or curl for port testing.
- 3 Test HTTP/HTTPS, TCP, and UDP as needed.
- 4 Test from EC2, on-prem, and cross-account.
- 5 Document results and isolate the issue.



Identify
Symptom



Map Path &
Architecture



Collect Logs &
Metrics



Analyze &
Isolate



Fix Configuration
or Policy



Validate &
Monitor

PRODUCTION NETWORKING TROUBLESHOOTING WORKFLOW



KEY POINTS

- ✓ Always understand the full traffic path.
- ✓ Use Flow Logs and Reachability Analyzer.
- ✓ Check NACLs before Security Groups.
- ✓ DNS issues can look like network issues.
- ✓ Test from multiple locations and components.
- ✓ Monitor and alert on network metrics.



YouTube
@veriqta



GitHub
@veriqta



LinkedIn
@veriqta



X
@veriqta



Instagram
@veriqta



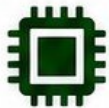
Telegram
@veriqta

32. PERFORMANCE TROUBLESHOOTING



Performance issues impact availability, user experience, and cost.

This page covers how to identify, analyze, and resolve performance bottlenecks across AWS environments.



1. CPU BOTTLENECKS

- High CPU utilization.
- Sustained CPU credits.
- CPU throttling.
- Inefficient code or processes.
- Too many context switches.

TROUBLESHOOTING STEPS

- 1 Check CloudWatch CPUUtilization.
- 2 Review CPU usage by process (top, htop).
- 3 Identify noisy neighbors (on shared hosts).
- 4 Analyze application threads and locks.
- 5 Scale up/out or optimize application.

KEY METRICS

CPUUtilization, CPUSurplusCredits, LoadAverage, ThrottledTime



2. MEMORY PRESSURE

- High memory usage.
- Swapping or disk paging.
- Out of memory (OOM) kills.
- Memory leaks.
- Insufficient instance memory.

TROUBLESHOOTING STEPS

- 1 Check CloudWatch MemoryUtilization.
- 2 Monitor swap usage (free -m, vmstat).
- 3 Review application memory settings.
- 4 Analyze heap dumps and memory leaks.
- 5 Increase memory or optimize workload.

KEY METRICS

MemoryUtilization, SwapUsage, PageFaults, OOMKills



3. STORAGE LATENCY

- High read/write latency.
- Low IOPS.
- Slow EBS or EFS performance.
- Disk saturation.
- Poor storage configuration.

TROUBLESHOOTING STEPS

- 1 Check CloudWatch VolumeLatency.
- 2 Review VolumeQueueLength and IOPS.
- 3 Run disk benchmarks (fio, iostat).
- 4 Verify EBS type and size.
- 5 Optimize file system and mount options.

KEY METRICS

VolumeLatency, VolumeIOPS, VolumeQueueLength, BurstBalance



4. NETWORK LATENCY

- High RTT between components.
- Packet loss.
- Bandwidth saturation.
- Poor routing.
- Cross-AZ or cross-region latency.

TROUBLESHOOTING STEPS

- 1 Check CloudWatch NetworkIn/Out.
- 2 Run ping, traceroute, mtr between endpoints.
- 3 Check VPC Flow Logs for drops.
- 4 Verify Security Groups and NACLs.
- 5 Optimize routing and placement.

KEY METRICS

NetworkIn, NetworkOut, PacketsDropped, TCPRetransmits, RTT



5. APPLICATION LATENCY

- Slow API responses.
- Database query delays.
- External service latency.
- Poor caching.
- Synchronous bottlenecks.

TROUBLESHOOTING STEPS

- 1 Use APM / X-Ray to find slow transactions.
- 2 Review p95/p99 latency in CloudWatch.
- 3 Analyze database slow query logs.
- 4 Check external dependencies.
- 5 Implement caching and async patterns.

KEY METRICS

Latency (p50/p95/p99), ErrorRate, DBLatency, APILatency



6. COST VS PERFORMANCE

- Over-provisioning resources.
- Under-provisioning workloads.
- Inefficient instance types.
- Unused or oversized storage.
- High data transfer costs.

TROUBLESHOOTING STEPS

- 1 Review AWS Cost Explorer and CUR.
- 2 Identify idle or underused resources.
- 3 Right-size instances and storage.
- 4 Use Savings Plans / Reserved Instances.
- 5 Balance performance and total cost.

KEY METRICS

Cost, Utilization, \$/Request, ResourceEfficiency



7. OPTIMIZATION STRATEGIES

- Right-size compute, memory, and storage.
- Use caching (ElastiCache, CloudFront, client-side).
- Optimize database indexes and queries.
- Enable compression and connection pooling.
- Use async processing and queues (SQS, Kinesis).

OPTIMIZATION CHECKLIST

- ✓ Identify bottleneck with data, not guesswork.
- ✓ Measure before and after changes.
- ✓ Use recommended AWS best practices.
- ✓ Automate scaling and resource management.
- ✓ Continuously monitor and improve.



Identify Symptom



Collect Metrics and Logs



Analyze Bottleneck



Apply Fix or Optimization



Validate & Monitor



KEY POINTS

- ✓ Always start with metrics and data.
- ✓ Focus on the biggest bottleneck first.
- ✓ Correlate across layers (Infra, Network, App).
- ✓ Small improvements compound over time.

- ✓ Balance performance, reliability, and cost.
- ✓ Automate alerts for performance metrics.
- ✓ Document findings and share knowledge.
- ✓ Keep revisiting and optimizing.



YouTube
@veriqta



GitHub
@veriqta



LinkedIn
@veriqta



X
@veriqta



Instagram
@veriqta



Telegram
@veriqta

33. SECURITY INCIDENT RESPONSE TROUBLESHOOTING



Quick, decisive action during a security incident limits impact and speeds up recovery.

This page covers common security incidents and the incident response process on AWS.



1. CREDENTIAL COMPROMISE

- Stolen access keys
- Leaked passwords
- Exposed secrets
- Session hijacking

RESPONSE STEPS

- 1 Revoke or deactivate keys.
- 2 Invalidate all active sessions.
- 3 Rotate passwords and secrets.
- 4 Review CloudTrail for activity.
- 5 Enforce MFA and least privilege.



2. IAM ABUSE

- Privilege escalation
- Unauthorized access
- Policy manipulation
- Creation of backdoors

RESPONSE STEPS

- 1 Identify and disable user/role.
- 2 Revoke excessive permissions.
- 3 Review and fix IAM policies.
- 4 Check for new users or roles.
- 5 Enable IAM Access Analyzer.



3. DATA EXPOSURE

- Public S3 buckets
- Unencrypted data
- Overly permissive access
- Sensitive data leakage

RESPONSE STEPS

- 1 Identify exposed resources.
- 2 Remove public access.
- 3 Restrict bucket and object access.
- 4 Enable encryption (SSE-KMS).
- 5 Monitor and review access logs.



4. MALWARE RESPONSE

- Malicious scripts
- Ransomware
- Crypto mining
- Persistence threats

RESPONSE STEPS

- 1 Isolate infected resources.
- 2 Capture disk/memory image.
- 3 Scan and identify malware.
- 4 Remove malware and artifacts.
- 5 Patch vulnerabilities.



5. ISOLATION

- Limit spread of attack
- Quarantine resources
- Block malicious traffic
- Contain compromised assets

RESPONSE STEPS

- 1 Isolate instance or workload.
- 2 Detach from network if needed.
- 3 Block IPs and malicious domains.
- 4 Update security groups and NACLs.
- 5 Monitor for lateral movement.



6. EVIDENCE COLLECTION

- Preserve forensic data
- Logs and configuration
- System snapshots
- Timeline reconstruction

RESPONSE STEPS

- 1 Enable and collect CloudTrail logs.
- 2 Take EBS and instance snapshots.
- 3 Export VPC Flow Logs and DNS logs.
- 4 Collect ALB, S3, and application logs.
- 5 Document timeline and actions taken.



7. RECOVERY

- Restore normal operations
- Validate integrity
- Lessons learned
- Strengthen security

RESPONSE STEPS

- 1 Remove compromised resources.
- 2 Restore from clean backups or images.
- 3 Rotate all credentials and secrets.
- 4 Validate systems and monitor closely.
- 5 Implement improvements and controls.

INCIDENT RESPONSE WORKFLOW



Identify Incident



Analyze Impact



Isolate Contain



Collect Evidence



Eradicate Threat



Recover Systems



Review & Improve



BEST PRACTICES

- ✓ Enable CloudTrail in all regions.
- ✓ Use least privilege and MFA everywhere.
- ✓ Regularly rotate credentials and secrets.
- ✓ Encrypt sensitive data by default.

- ✓ Monitor with GuardDuty, Security Hub, and CloudWatch.
- ✓ Test incident response with tabletop exercises.
- ✓ Keep backups, runbooks, and contacts ready.
- ✓ Review and improve security continuously.



YouTube
@veriqta



GitHub
@veriqta



LinkedIn
@veriqta



X
@veriqta



Instagram
@veriqta



Telegram
@veriqta

34. DISASTER RECOVERY TROUBLESHOOTING



Disaster recovery ensures business continuity during failures and outages.

This page covers common DR issues and how to troubleshoot, validate, and recover AWS environments.

1. BACKUP VALIDATION



- Failed backups
- Incomplete backups
- Corrupted backups
- Retention misconfiguration
- Permissions issues

TROUBLESHOOTING STEPS

- 1 Verify backup job status and logs.
- 2 Check backup completeness.
- 3 Validate restore points.
- 4 Test restore to isolated environment.
- 5 Review retention and lifecycle rules.

2. CROSS-REGION RECOVERY



- Replication lag
- Data synchronization issues
- DNS failover problems
- Route 53 misconfiguration
- Service dependency failures

TROUBLESHOOTING STEPS

- 1 Check replication health and log.
- 2 Verify cross-region configurations.
- 3 Validate Route 53 failover policies.
- 4 Test application in DR region.
- 5 Review IAM and network access.

3. MULTI-AZ FAILURES



- AZ impairments
- Failover not occurring
- Unhealthy instances
- Load balancer issues
- Database failover failures

TROUBLESHOOTING STEPS

- 1 Check AZ health in AWS Health.
- 2 Verify Auto Scaling group health.
- 3 Check ELB target health.
- 4 Validate RDS/ElastiCache failover.
- 5 Review subnet and route tables.

4. REGION OUTAGES



- Service unavailability
- Control plane issues
- Network disruptions
- API/Endpoint failures
- SLA impact

TROUBLESHOOTING STEPS

- 1 Check AWS Health Dashboard.
- 2 Identify affected services.
- 3 Fail over to DR region.
- 4 Verify data consistency.
- 5 Communicate and escalate.

5. RESTORE TESTING



- Restore failures
- Slow restore speed
- Missing dependencies
- Data integrity issues
- Configuration drift

TROUBLESHOOTING STEPS

- 1 Review restore job logs.
- 2 Verify backup integrity.
- 3 Check dependent resources.
- 4 Validate data after restore.
- 5 Document issues and fixes.

6. RTO/RPO VALIDATION



- RTO not achievable
- RPO violations
- Poor recovery performance
- Incorrect assumptions
- Monitoring gaps

TROUBLESHOOTING STEPS

- 1 Review RTO and RPO requirements.
- 2 Measure actual recovery times.
- 3 Compare against business targets.
- 4 Optimize architecture and automation.
- 5 Update documentation and plans.

7. RECOVERY DRILLS



- Drill failures
- Unclear runbooks
- Team unpreparedness
- Communication gaps
- Tooling issues

TROUBLESHOOTING STEPS

- 1 Review drill results and gaps.
- 2 Update and simplify runbooks.
- 3 Train teams and assign roles.
- 4 Automate repeatable tasks.
- 5 Schedule regular DR drills.

DISASTER RECOVERY LIFECYCLE



Identify Risk



Plan Recovery



Back Up & Replicate



Test & Validate



Recover & Fail Over



Review & Improve



BEST PRACTICES

- Follow the 3-2-1 backup strategy.
- Automate backups and recovery tasks.
- Test restores regularly, not just backups.
- Document runbooks and dependencies.
- Use cross-region replication for critical data.
- Monitor recovery metrics and alerts.
- Keep runbooks, contacts, and credentials updated.
- Continuously improve and reduce RTO/RPO.



YouTube
@veriqta



GitHub
@veriqta



LinkedIn
@veriqta



X
@veriqta



Instagram
@veriqta



Telegram
@veriqta

35. PRODUCTION OUTAGE CASE STUDIES

TROUBLESHOOTING



Real production outages happen.

Studying past incidents helps us troubleshoot faster and prevent future failures. These case studies show causes, impact, detection, response, and recovery timelines.



1. COMPLETE WEBSITE OUTAGE

Cause: ALB target group misconfiguration
Impact: 100% of users affected
Detection: CloudWatch alarms
Duration: 47 minutes
Root Cause: Wrong health check path

RECOVERY TIMELINE

- ✓ 00:00 Users report outage
- ✓ 00:03 Alarm triggered
- ✓ 00:07 Issue identified
- ✓ 00:18 Fix deployed
- ✓ 00:47 Full recovery

KEY TAKEAWAYS

- ✓ Validate health checks after changes
- ✓ Use canary checks and synthetic monitoring
- ✓ Automate rollback for critical services



2. DATABASE OUTAGE

Cause: RDS primary instance failure
Impact: Application errors, writes failed
Detection: CloudWatch + errors
Duration: 1 hour 23 minutes
Root Cause: Storage corruption

RECOVERY TIMELINE

- ✓ 00:00 High error rate detected
- ✓ 00:04 RDS failure identified
- ✓ 00:12 Failover initiated
- ✓ 00:45 Application reconnects
- ✓ 01:23 Full recovery

KEY TAKEAWAYS

- ✓ Multi-AZ is not a backup
- ✓ Test failover and backups regularly
- ✓ Monitor storage and disk health



3. DNS FAILURE

Cause: Route 53 record misconfiguration
Impact: Intermittent website failures
Detection: User reports + monitoring
Duration: 36 minutes
Root Cause: Incorrect alias target

RECOVERY TIMELINE

- ✓ 00:00 Users report errors
- ✓ 00:02 DNS resolution issues found
- ✓ 00:08 Bad record identified
- ✓ 00:16 Record update deployed
- ✓ 00:36 Full recovery

KEY TAKEAWAYS

- ✓ Use change validation before updates
- ✓ Enable DNS health checks
- ✓ Monitor resolution from multiple regions



4. NETWORK ISOLATION

Cause: Security group rule removed
Impact: Services unreachable
Detection: Synthetic checks failed
Duration: 55 minutes
Root Cause: Overly restrictive SG rule

RECOVERY TIMELINE

- ✓ 00:00 Connectivity alerts
- ✓ 00:05 Reachability tests failed
- ✓ 00:12 SG change identified
- ✓ 00:25 Rule restored
- ✓ 00:55 Full recovery

KEY TAKEAWAYS

- ✓ Use infrastructure as code
- ✓ Test connectivity after security changes
- ✓ Document and review network rules



5. DEPLOYMENT FAILURE

Cause: New version had memory leak
Impact: High CPU, service crash
Detection: CloudWatch alarms
Duration: 1 hour 5 minutes
Root Cause: Insufficient load testing

RECOVERY TIMELINE

- ✓ 00:00 CPU alarms triggered
- ✓ 00:04 Deployment identified
- ✓ 00:10 Rollback initiated
- ✓ 00:22 Previous version restored
- ✓ 01:05 Full recovery

KEY TAKEAWAYS

- ✓ Use blue/green or canary deployments
- ✓ Load test before production release
- ✓ Implement automated rollback



6. AUTHENTICATION OUTAGE

Cause: Cognito user pool misconfig
Impact: Login and token failures
Detection: Error rate + user reports
Duration: 42 minutes
Root Cause: Invalid app client secret

RECOVERY TIMELINE

- ✓ 00:00 Login failures detected
- ✓ 00:03 Cognito errors identified
- ✓ 00:07 Wrong secret found
- ✓ 00:18 Secret updated
- ✓ 00:42 Full recovery

KEY TAKEAWAYS

- ✓ Store secrets securely
- ✓ Monitor auth service health
- ✓ Test auth flows after changes

OUTAGE RESPONSE PRINCIPLES



Detect
Early



Assess
Impact



Communicate
Clearly



Act
Fast



Document
Everything



Learn &
Improve



BEST PRACTICES

- ✓ Monitor everything that matters
- ✓ Automate alerts and incident detection
- ✓ Have a runbook for every critical service
- ✓ Test recovery regularly, not just backups
- ✓ Communicate with stakeholders early
- ✓ Keep runbooks, contacts, and diagrams updated
- ✓ Review incidents and improve continuously
- ✓ Design for failure: Multi-AZ, Multi-Region, Backups



YouTube
@veriqta



GitHub
@veriqta



LinkedIn
@veriqta



X
@veriqta



Instagram
@veriqta



Telegram
@veriqta

36. INCIDENT RESPONSE PLAYBOOKS



Incident response playbooks provide a repeatable, structured approach to detect, respond to, and recover from incidents.

Use these playbooks to act fast, reduce impact, and learn from every incident.

1. DETECTION



- Monitor systems, logs, and metrics
- Alerts and anomaly detection
- User or customer reports

ACTIONS

- 1 Monitor dashboards and alerts
- 2 Review logs and metrics
- 3 Validate the alert
- 4 Identify affected resources
- 5 Create incident record

KEY OUTPUTS

- Incident detected
- Initial impact identified
- Incident record created

2. TRIAGE



- Determine severity and impact
- Identify scope
- Decide response priority

ACTIONS

- 1 Assess severity (Sev 1-4)
- 2 Identify affected users and services
- 3 Determine root cause hypothesis
- 4 Decide next steps
- 5 Assign incident owner

KEY OUTPUTS

- Severity and priority set
- Incident owner assigned
- Triage notes captured

3. ESCALATION



- Escalate based on severity
- Notify the right teams
- Engage experts early

ACTIONS

- 1 Follow escalation matrix
- 2 Notify on-call engineers
- 3 Engage subject matter experts
- 4 Update stakeholders
- 5 Open bridge / war room

KEY OUTPUTS

- Right team engaged
- Stakeholders notified
- War room established

4. COMMUNICATION



- Keep stakeholders informed
- Provide clear, timely updates
- Avoid information gaps

ACTIONS

- 1 Send initial notification
- 2 Provide status updates
- 3 Share impact and ETA
- 4 Communicate in channels
- 5 Document all updates

KEY OUTPUTS

- Stakeholders informed
- Consistent updates
- Communication log

5. RESOLUTION



- Implement fix
- Mitigate impact
- Restore normal service

ACTIONS

- 1 Implement mitigation
- 2 Apply permanent fix
- 3 Verify service restoration
- 4 Monitor closely
- 5 Communicate resolution

KEY OUTPUTS

- Service restored
- Root cause addressed
- Monitoring in place

6. VALIDATION



- Confirm system stability
- Verify data integrity
- Ensure no side effects

ACTIONS

- 1 Run health checks
- 2 Validate system behavior
- 3 Verify data integrity
- 4 Monitor for anomalies
- 5 Get user confirmation

KEY OUTPUTS

- System stable
- No related issues
- Validation evidence

7. POST-INCIDENT REVIEW



- Analyze what happened
- Identify improvements
- Update playbooks and runbooks

ACTIONS

- 1 Conduct blameless review
- 2 Document timeline
- 3 Identify root causes
- 4 Define corrective actions
- 5 Update playbooks and docs

KEY OUTPUTS

- Lessons learned
- Action items created
- Playbooks improved

INCIDENT RESPONSE WORKFLOW



Detect



Triage



Escalate



Communicate



Resolve



Validate



Review



BEST PRACTICES

- ✓ Detect early and act fast.
- ✓ Follow the escalation path.
- ✓ Communicate proactively.
- ✓ Document every action.

- ✓ Validate fully before closing.
- ✓ Learn from every incident.
- ✓ Continuously improve playbooks.
- ✓ Run regular simulations and drills.



YouTube
@veriqta



GitHub
@veriqta



LinkedIn
@veriqta



X
@veriqta



Instagram
@veriqta



Telegram
@veriqta

37. AWS SUPPORT TOOLS

TROUBLESHOOTING



AWS provides powerful built-in tools and support services to help you monitor, troubleshoot, and resolve issues faster.

Use the right tool at the right time to save time and minimize impact.

1. SYSTEMS MANAGER



- Operational visibility
- Automation
- Configuration management
- Remote access

USE CASES

- Troubleshoot EC2 & on-prem
- Run commands at scale
- Patch and manage instances
- Collect logs and metrics

KEY FEATURES

- Session Manager
- Run Command
- State Manager
- Automation
- Parameter Store
- Systems Manager Inventory

2. INCIDENT MANAGER



- Centralized incident management
- Response coordination
- Timeline & communication

USE CASES

- Manage operational incidents
- Track incident lifecycle
- Collaborate with teams
- Integrate with chat/ITSM

KEY FEATURES

- Incident record
- Response plans
- ChatOps integration
- Timeline & actions
- Metrics & impact tracking
- Post-incident review

3. TRUSTED ADVISOR



- Best practice checks
- Cost optimization
- Security
- Fault tolerance
- Performance

USE CASES

- Identify configuration issues
- Improve security posture
- Reduce costs
- Improve reliability

KEY FEATURES

- 5 Pillars: Cost, Security, Fault Tolerance, Performance, Service Limits
- Real-time recommendations
- Actionable guidance

4. HEALTH DASHBOARD



- Service health status
- Scheduled events
- Real-time issues
- Regional visibility

USE CASES

- Check AWS service health
- Investigate ongoing issues
- Track events & changes
- Validate recovery

KEY FEATURES

- Personal Health Dashboard
- Service Health Dashboard
- Event notifications
- Historical events
- RSS/Email updates

5. AWS SUPPORT AUTOMATION



- Automated diagnostics
- Pre-built runbooks
- Self-healing actions
- Reduce MTTR

USE CASES

- Automate common remediation
- Run diagnostics automatically
- Trigger actions based on events
- Reduce manual effort

KEY FEATURES

- AWSsupport-Troubleshoot* runbooks
- EventBridge + Automation
- Integration with Systems Manager
- Custom runbooks

6. AWS CLI



- Command-line access
- Scripting & automation
- API control
- Debug & test

USE CASES

- Query & inspect resources
- Automate tasks
- Collect logs & metrics
- Reproduce & debug issues

KEY FEATURES

- Comprehensive service coverage
- Output in JSON / YAML / Table
- Profiles & regions
- Paginator & filters
- JMESPath queries

7. SDK TROUBLESHOOTING



- Programmatic access
- Error handling
- Retry & backoff
- Debug API calls

USE CASES

- Troubleshoot application errors
- Handle throttling & retries
- Debug API requests
- Improve reliability

KEY FEATURES

- Detailed error responses
- Retry & backoff strategies
- Logging & tracing
- Request IDs & metrics
- Service-specific exceptions

HOW TO CHOOSE THE RIGHT TOOL



Identify
The Issue



Understand
The Impact



Choose The
Right Tool



Run Diagnostics
& Fix



Validate &
Monitor



Document &
Improve



BEST PRACTICES

- ✓ Use built-in tools before opening a support case.
- ✓ Automate diagnostics and remediation.
- ✓ Monitor proactively and set alerts.
- ✓ Keep runbooks and documentation current.
- ✓ Use CLI/SDK for automation and repeatable tasks.
- ✓ Validate fixes and confirm full recovery.
- ✓ Review incidents and improve continuously.
- ✓ Leverage AWS Support plans for faster response.



YouTube
@veriqta



GitHub
@veriqta



LinkedIn
@veriqta



X
@veriqta



Instagram
@veriqta



Telegram
@veriqta

38. PRODUCTION TROUBLESHOOTING LABS

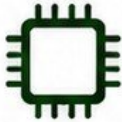
TROUBLESHOOTING



Hands-on labs help you practice real-world troubleshooting skills in a safe environment.

Each lab simulates a production incident and guides you through detection, analysis, and resolution.

1. EC2 OUTAGE LAB



SCENARIO

An EC2 instance becomes unreachable and the application is down.

OBJECTIVES

- Identify the root cause
- Restore instance connectivity
- Validate service recovery

KEY SKILLS

- EC2 troubleshooting
- System status checks
- Security group analysis
- CloudWatch logs

2. RDS FAILURE LAB



SCENARIO

An RDS database is unavailable due to failover or configuration issues.

OBJECTIVES

- Diagnose database failure
- Perform failover or recovery
- Validate data availability

KEY SKILLS

- RDS monitoring
- Failover and snapshots
- Parameter group analysis
- Performance insights

3. EKS INCIDENT LAB



SCENARIO

Applications in EKS are failing due to node or pod issues.

OBJECTIVES

- Identify cluster or workload issue
- Restore application health
- Validate cluster stability

KEY SKILLS

- EKS debugging
- Kubernetes events & logs
- Pod & deployment analysis
- Resource & autoscaling

4. IAM FAILURE LAB



SCENARIO

Users cannot access AWS resources due to IAM policy or role issues.

OBJECTIVES

- Identify permission problem
- Fix IAM policy or role
- Validate access restoration

KEY SKILLS

- IAM policies & roles
- Trust relationships
- Access Analyzer
- Permission boundaries

5. NETWORK OUTAGE LAB



SCENARIO

Resources in a VPC cannot reach the internet or other services.

OBJECTIVES

- Diagnose connectivity issue
- Fix routing, ACL or SG issues
- Validate full connectivity

KEY SKILLS

- VPC troubleshooting
- Route tables & NACLs
- Security groups
- VPC Flow Logs

6. CI/CD FAILURE LAB



SCENARIO

A pipeline fails during build or deployment.

OBJECTIVES

- Identify pipeline failure cause
- Fix build or deploy issue
- Achieve successful release

KEY SKILLS

- Pipeline logs analysis
- Artifact & permission issues
- Service integration
- Rollback strategies

7. DISASTER RECOVERY LAB



SCENARIO

Primary region is down and services must be recovered in DR region.

OBJECTIVES

- Failover to DR environment
- Restore data and services
- Validate business continuity

KEY SKILLS

- Cross-region replication
- Route 53 failover
- Backup & restore
- RTO/RPO validation



LAB BENEFITS

- ✓ Build real troubleshooting confidence
- ✓ Learn by doing, not just theory
- ✓ Understand AWS services deeply
- ✓ Improve incident response speed
- ✓ Practice in safe, isolated environments
- ✓ Prepare for real production incidents
- ✓ Strengthen problem-solving skills
- ✓ Document learnings for future use



YouTube
@veriqta



GitHub
@veriqta



LinkedIn
@veriqta



X
@veriqta



Instagram
@veriqta



Telegram
@veriqta

39. ENTERPRISE TROUBLESHOOTING CHECKLIST

TROUBLESHOOTING



Use this enterprise checklist to investigate, resolve, and recover from production issues.

Follow the workflow. Document everything.
Learn and improve continuously.

1 INVESTIGATION WORKFLOW



- ✓ Define the problem clearly
- ✓ Identify impact and scope
- ✓ Collect logs and metrics
- ✓ Establish timeline
- ✓ Identify recent changes
- ✓ Form a hypothesis
- ✓ Test the hypothesis
- ✓ Determine root cause
- ✓ Document findings

2 ESCALATION MATRIX



- ✓ Level 1: On-call engineer
- ✓ Level 2: Team lead / SRE
- ✓ Level 3: Engineering manager
- ✓ Level 4: Director / Head
- ✓ Level 5: Executive team
- ✓ Define escalation triggers
- ✓ Include contact information
- ✓ Define communication path
- ✓ Escalate early, not late

3 DOCUMENTATION



- ✓ Record incident timeline
- ✓ Capture symptoms
- ✓ Document actions taken
- ✓ Record commands used
- ✓ Attach logs and screenshots
- ✓ Note communications
- ✓ Update incident status
- ✓ Store in knowledge base
- ✓ Keep documentation clean

4 MONITORING CHECKLIST



- ✓ Check system health
- ✓ Review dashboards
- ✓ Validate alerts
- ✓ Check logs for errors
- ✓ Verify dependencies
- ✓ Check resource utilization
- ✓ Review SLAs / SLOs
- ✓ Validate synthetic checks
- ✓ Confirm alert routing

5 RECOVERY CHECKLIST



- ✓ Implement fix or workaround
- ✓ Restore affected services
- ✓ Validate functionality
- ✓ Monitor system stability
- ✓ Check for side effects
- ✓ Scale resources if needed
- ✓ Confirm user access
- ✓ Update status page
- ✓ Communicate resolution

6 VERIFICATION CHECKLIST



- ✓ Verify issue is fully resolved
- ✓ Run validation tests
- ✓ Check all user flows
- ✓ Validate performance
- ✓ Confirm monitoring is normal
- ✓ Ensure no error logs
- ✓ Verify dependencies
- ✓ Confirm SLIs / SLOs
- ✓ Get final confirmation

7 LESSONS LEARNED



- ✓ What happened?
- ✓ What was the root cause?
- ✓ What went well?
- ✓ What did not go well?
- ✓ What can be improved?
- ✓ Update runbooks
- ✓ Update monitoring / alerts
- ✓ Share knowledge with team
- ✓ Prevent recurrence



BEST PRACTICES

- ✓ Stay calm and follow the process
- ✓ Communicate clearly and often
- ✓ Document everything in real time
- ✓ Use data, not assumptions

- ✓ Automate repetitive diagnostics
- ✓ Review and improve runbooks
- ✓ Run post-incident reviews
- ✓ Continuously improve systems

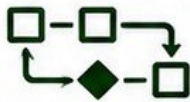
40. SENIOR CLOUD ENGINEER FINAL REVIEW

TROUBLESHOOTING



This is your final review. You now have the skills, tools, processes, and mindset to solve real production problems. Use this framework to troubleshoot anything, anywhere, and lead with confidence.

1 END-TO-END TROUBLESHOOTING WORKFLOW



- 1 Detect the issue
- 2 Gather information
- 3 Narrow down the scope
- 4 Identify the root cause
- 5 Implement the fix
- 6 Verify the resolution
- 7 Document everything
- 8 Learn and improve

2 ENTERPRISE PRODUCTION CHECKLIST



- 2 Monitoring and alerting in place
- 2 Runbooks and playbooks available
- 3 Backups tested and verified
- 4 Security controls validated
- 5 Scaling and capacity reviewed
- 6 Disaster recovery validated
- 7 Dependencies documented
- 8 Change management followed

3 COMMON FAILURE PATTERNS



- ✓ Resource limits (CPU, memory, disk, connections)
- ✓ Misconfigurations
- ✓ Network and security issues
- ✓ Dependency or upstream failures
- ✓ IAM and permission problems
- ✓ Performance bottlenecks
- ✓ Deployment and configuration drift

4 SENIOR ENGINEER MINDSET



- ✓ Stay calm and think clearly
- ✓ Ask the right questions
- ✓ Verify, do not assume
- ✓ Use data, not opinions
- ✓ Consider the customer impact
- ✓ Communicate clearly
- ✓ Own the problem
- ✓ Focus on prevention

5 CONTINUOUS IMPROVEMENT



- ✓ Post-incident reviews
- ✓ Update runbooks and playbooks
- ✓ Automate repetitive tasks
- ✓ Improve monitoring and alerts
- ✓ Reduce MTTR and increase reliability
- ✓ Share knowledge with the team
- ✓ Drive a culture of learning

6 LEARNING ROADMAP



CORE FOUNDATIONS

Linux • Networking • Security
Cloud Services • Databases

ADVANCED SKILLS

Automation • Observability
Performance • Reliability
Architecture • Cost Optimization

LEADERSHIP SKILLS

Communication • Mentoring
Problem Solving • Decision Making
Ownership • Business Impact

Never stop learning.
The cloud is always evolving.

7 PRODUCTION READINESS REVIEW



- ✓ System is highly available
- ✓ Monitoring and alerts are active
- ✓ Backups are tested
- ✓ Security is hardened
- ✓ Performance is validated
- ✓ Scalability is confirmed
- ✓ DR plan is validated
- ✓ Runbooks are ready
- ✓ Team is prepared
- ✓ Go / No-Go confirmed

8 FINAL WORD



You are now equipped to handle real-world production incidents at any scale.

**SOLVE PROBLEMS.
IMPROVE SYSTEMS.
MAKE AN IMPACT.
LEAD WITH EXCELLENCE.**

SENIOR ENGINEER TROUBLESHOOTING WORKFLOW



Detect the Issue



Gather Information



Identify Root Cause



Implement the Fix



Verify Resolution



Document Everything



Learn and Improve



BEST PRACTICES

- ✓ Follow the process every time.
- ✓ Use the right tools.
- ✓ Verify before you assume.
- ✓ Document everything.
- ✓ Communicate early and often.
- ✓ Automate and standardize.
- ✓ Learn from every incident.
- ✓ Make reliability everyone's responsibility.



YouTube
@veriqta



GitHub
@veriqta



LinkedIn
@veriqta



X
@veriqta



Instagram
@veriqta



Telegram
@veriqta